

```
// server.js - Backend nối dashboard với Meta Marketing API
// HỖ TRỢ NHIỀU DOANH NGHIỆP (TOKEN_1, TOKEN_2, ...) + NHẬN DIỆN TÊN NHÂN VIÊN
// Yêu cầu Node.js 18 trở lên. Chạy: npm install → npm start → mở http://localhost
```

```
import express from 'express';
import session from 'express-session';
import 'dotenv/config';
import path from 'node:path';
import fs from 'node:fs';
import crypto from 'node:crypto';
import { fileURLToPath } from 'node:url';
```

```
const __dirname = path.dirname(fileURLToPath(import.meta.url));
```

```
// — fetchWithTimeout: tự huỷ request sau N giây để tránh treo vô hạn —
```

```
const FETCH_TIMEOUT_MS = 30000; // 30 giây
```

```
function fetchWithTimeout(url, opts = {}, timeoutMs = FETCH_TIMEOUT_MS) {
  const ctrl = new AbortController();
  const timer = setTimeout(() => ctrl.abort(), timeoutMs);
  return fetch(url, { ...opts, signal: ctrl.signal })
    .finally(() => clearTimeout(timer));
}
```

```
const app = express();
```

```
// Thư mục lưu dữ liệu NGOÀI project (không bị mất khi deploy). Khai báo SỚM
```

```
// vì nhiều phần bên dưới dùng tới. __dirname = .../ads.tdmjsc.com/nodejs
```

```
const DATA_DIR = process.env.DATA_DIR || path.join(__dirname, '..', '..', '..');
try { fs.mkdirSync(DATA_DIR, { recursive: true }); } catch (e) {}
```

```
const V = process.env.META_API_VERSION || 'v23.0';
```

```
const BASE = `https://graph.facebook.com/${V}`;
```

```
/* =====
```

```
  TÀI KHOẢN ĐĂNG NHẬP
```

```
  Danh sách tài khoản nằm ở FILE RIÊNG ./users.js do BẠN tự quản lý.
```

```
  → Cập nhật server.js sẽ KHÔNG ghi đè tài khoản của bạn nữa.
```

```
  → Bạn tự điền mật khẩu trong users.js (không cần gửi cho ai).
```

```
  Nếu chưa có file users.js, hệ thống dùng tạm 1 tài khoản admin mặc định.
```

```
  ===== */
```

```
let USERS = [{ user: 'admin', pass: 'DOI_MAT_KHAU_NAY', role: 'admin' }];
```

```
let TELEGRAM_CHAT_IDS = {};
```

```
import('./users.js')
```

```
  .then(mod => {
```

```
    if (Array.isArray(mod.USERS) && mod.USERS.length) USERS = mod.USERS;
```

```

    if (mod.TELEGRAM_CHAT_IDS) TELEGRAM_CHAT_IDS = mod.TELEGRAM_CHAT_IDS;
  })
  .catch(() => console.log('Chưa tìm thấy users.js – đang dùng tài khoản admin mặ

/* =====
DANH SÁCH NHÂN VIÊN ← BẠN ĐIỀN PHẦN NÀY
Điền tên tất cả nhân viên chạy quảng cáo. Hệ thống sẽ tìm tên này
XUẤT HIỆN trong tên chiến dịch, dù viết kiểu nào:
    "Phuong- Balo..."    "29/3 Phuong"    "Huân Máy cho cá ăn" → đều nhận ra.
- Tên có dấu hay không đều khớp (Phuong = Phuong).
- Nếu hai người tên gần giống nhau, ghi đầy đủ hơn (vd "Việt Hà").
===== */

const EMPLOYEES = [
  // { code: mã đứng đầu tên chiến dịch, short: tên ngắn (chiến dịch cũ), full: t
  // bnhx: số tiền BHXH mặc định hàng tháng (điền 0 nếu không có)
  { code: 'TD1', short: 'Trường', full: 'Tạ Quang Trường', bnhx: 598500 },
  { code: 'TD2', short: 'Phuong', full: 'Trịnh Đức Phuong', bnhx: 598500 },
  { code: 'TD3', short: 'Hiệu', full: 'Nguyễn Trung Hiệu', bnhx: 577500 },
  { code: 'TD4', short: 'My', full: 'Nguyễn Thị Trà My', bnhx: 577500 },
  { code: 'TD5', short: 'Ánh', full: 'Lê Thị Ánh', bnhx: 577500 },
  { code: 'TD6', short: 'Huân', full: 'Nguyễn Duy Huân', bnhx: 0 },
  { code: 'TD7', short: 'Minh', full: 'Dương Văn Minh', bnhx: 577500 },
  { code: 'TD8', short: 'Giang', full: 'Vũ Hà Giang', bnhx: 577500 },
  { code: 'TD9', short: 'Việt Hà', full: 'Đoàn Việt Hà', bnhx: 577500 },
  { code: 'TD11', short: 'Tuyết', full: 'Lê Thị Ánh Tuyết', bnhx: 0 },
  { code: 'TD12', short: 'Tùng', full: 'Phạm Quang Tùng', bnhx: 0 },
  { code: 'TD13', short: 'Phong', full: 'Luu Xuân Phong', bnhx: 0 },
  { code: 'TD14', short: 'Thu Phuong', full: 'Nguyễn Thị Thu Phuong', bnhx:
  { code: 'TD10', short: 'Thúy An', full: 'Vũ Thúy An', aliases: ['Thúy An'], bnhx:
  // Nhân viên cũ (chiến dịch cũ chưa có mã) – giữ để vẫn nhận ra, xóa nếu không
  { code: '', short: 'Thắng', full: 'Thắng', bnhx: 0 },
];

/* Nhận diện nhân viên: tách tên chiến dịch thành các "chữ" riêng, rồi tìm xem
tên nhân viên có xuất hiện trọn vẹn như một (hoặc vài) chữ liền nhau không.
- KHỚP CÓ DẤU: "Mỹ" sẽ KHÔNG bị tính thành "My"; "hiệu" không thành "Hiệu".
- Tên 2 chữ như "Thúy An", "Việt Hà" được ưu tiên khớp trước.
- Lưu ý: nhân viên phải viết đúng dấu tên mình trong tên chiến dịch.
    Nếu viết sai/thiếu dấu, chiến dịch đó sẽ rơi vào nhóm "Chưa xác định"
    để bạn nhìn thấy và sửa. */

function detectEmployee(name) {
  const raw = (name || '').split(/[^\p{L}\p{N}]+/u).filter(Boolean);

  // 1) Ưu tiên MÃ ở token ĐẦU TIÊN, vd "TD1-Trường-..." (chiến dịch từ T6 trở đi
  if (raw.length) {
    const code = raw[0].toUpperCase();
    const byCode = EMPLOYEES.find(e => e.code && e.code.toUpperCase() === code);

```

```

    if (byCode) return byCode.full;
}

// 2) Không có mã -> nhận diện theo TÊN NGẮN như cũ (khớp CÓ DẤU, ưu tiên tên n
const tokens = raw.map(t => t.toLowerCase());
const candidates = [];
for (const e of EMPLOYEES) {
    for (const nm of [e.short, ...(e.aliases || [])]) {
        if (nm) candidates.push({ words: nm.toLowerCase().trim().split(/\s+/), full
    }
}
candidates.sort((a, b) => b.words.length - a.words.length);
for (const c of candidates) {
    const w = c.words;
    for (let i = 0; i + w.length <= tokens.length; i++) {
        let ok = true;
        for (let j = 0; j < w.length; j++) {
            if (tokens[i + j] !== w[j]) { ok = false; break; }
        }
        if (ok) return c.full;
    }
}
return 'Chưa xác định';
}

/* ----- Đọc danh sách doanh nghiệp từ .env ----- */
const SOURCES = [];
let n = 1;
while (process.env[`TOKEN_${n}`]) {
    SOURCES.push({
        token: process.env[`TOKEN_${n}`],
        accounts: (process.env[`ACCOUNTS_${n}`] || '').split(',').map(s => s.trim()).
    });
    n++;
}
if (SOURCES.length === 0 && process.env.META_ACCESS_TOKEN) {
    SOURCES.push({
        token: process.env.META_ACCESS_TOKEN,
        accounts: (process.env.AD_ACCOUNT_IDS || '').split(',').map(s => s.trim()).fi
    });
}

const ACCOUNT_NAMES = {
    // Tên hiển thị cho từng tài khoản quảng cáo: 'ID': 'Tên'
    // (sửa tên bên phải dấu : nếu cần, GIỮ nguyên ID bên trái)
    '513728869887825': 'BM TD2.11',
    '635675708897994': 'BM TD2.22',

```

```

'1297945788377836': 'BM 3.1',
'3313861842124068': 'BM 3.2',
'1343265570424604': 'BM 3.3',
'932875756194538': 'BM 1.1',
'1257974532611757': 'BM 1.2',
'1460178049143518': 'BM 1.3',
'927174399921442': 'BM 1.4',
'4353143571590707': 'BM 1.5',
'911948588381996': 'BM 5.1',
'1050285000893195': 'BM 5.2',
'2535220170264046': 'BM 5.3',
};

```

```

const RESULT_RULES = [
  { match: o => /MESSAGE|ENGAGEMENT/.test(o), label: 'tin nhắn',
    types: ['onsite_conversion.messaging_conversation_started_7d', 'onsite_conver
  { match: o => /SALES|CONVERSION/.test(o), label: 'đơn hàng',
    types: ['omni_purchase', 'purchase', 'offsite_conversion.fb_pixel_purchase']
  { match: o => /LEAD/.test(o), label: 'lead',
    types: ['lead', 'onsite_conversion.lead_grouped', 'offsite_conversion.fb_pixe
  { match: () => true, label: 'kết quả',
    types: ['link_click', 'landing_page_view'] },
];

```

```

function countResults(actions = [], objective = '') {
  const rule = RESULT_RULES.find(r => r.match(objective)) || RESULT_RULES[RESULT_
  for (const t of rule.types) {
    const hit = actions.find(a => a.action_type === t);
    if (hit) return { value: Number(hit.value) || 0, label: rule.label };
  }
  return { value: 0, label: rule.label };
}

```

```

async function fb(endpoint, params, token) {
  const url = new URL(`${BASE}/${endpoint}`);
  url.searchParams.set('access_token', token);
  for (const [k, v] of Object.entries(params || {}))
    url.searchParams.set(k, typeof v === 'string' ? v : JSON.stringify(v));
  const res = await fetch(url);
  const json = await res.json();
  if (json.error) throw new Error(`Meta API: ${json.error.message}`);
  return json;
}

```

```

async function fbAll(endpoint, params, token) {
  let data = await fb(endpoint, params, token);
  let out = data.data || [];
  while (data.paging && data.paging.next) {
    const res = await fetch(data.paging.next);

```

```

    data = await res.json();
    if (data.error) throw new Error(`Meta API: ${data.error.message}`);
    out = out.concat(data.data || []);
  }
  return out;
}

function listDays(since, until) {
  const days = [];
  for (let d = new Date(since); d <= new Date(until); d.setDate(d.getDate() + 1))
    days.push(d.toISOString().slice(0, 10));
  return days;
}

async function fetchAccount(acc, token, days, since, until) {
  const accName = ACCOUNT_NAMES[acc] || `TK ${acc}`;
  const camps = await fbAll(`act_${acc}/campaigns`, {
    fields: 'id,name,status,effective_status,daily_budget,lifetime_budget,objecti
    limit: 500,
  }, token);

  const byId = {};
  for (const c of camps) {
    byId[c.id] = {
      acc, accName,
      id: c.id,
      name: c.name,
      employee: detectEmployee(c.name),
      on: c.effective_status === 'ACTIVE',
      objective: c.objective || '',
      budget: Number(c.daily_budget || c.lifetime_budget || 0),
      daily: days.map(() => ({ spent: 0, results: 0 })),
      obj: 'kết quả',
      link: null,
      adsets: [],
    };
  }
}

// Ba nhóm truy vấn dưới đây độc lập nhau -> chạy SONG SONG cho nhanh.
await Promise.all([

  // (1) Chi tiêu + kết quả theo từng ngày (cấp chiến dịch)
  (async () => {
    try {
      const insights = await fbAll(`act_${acc}/insights`, {
        level: 'campaign',
        fields: 'campaign_id,spend,actions,date_start',

```

```

        time_range: { since, until },
        time_increment: 1,
        limit: 1000,
    }, token);
    for (const row of insights) {
        const camp = byId[row.campaign_id];
        if (!camp) continue;
        const idx = days.indexOf(row.date_start);
        if (idx < 0) continue;
        const r = countResults(row.actions, camp.objective);
        camp.daily[idx] = { spent: Number(row.spend || 0), results: r.value };
        camp.obj = r.label;
    }
    } catch (e) { /* bỏ qua, các số liệu khác vẫn hiển thị */ }
    })(),

// (2) Link bài quảng cáo (cấp ad), ưu tiên ad đang chạy
(async () => {
    try {
        const ads = await fbAll(`act_${acc}/ads`, {
            fields: 'campaign_id,effective_status,preview_shareable_link,creative{
            limit: 500,
        }, token);
        const chosen = {};
        for (const ad of ads) {
            const cid = ad.campaign_id;
            if (!cid || !byId[cid]) continue;
            const link = (ad.creative && ad.creative.effective_object_story_id
                ? `https://www.facebook.com/${ad.creative.effective_object_stor
                || ad.preview_shareable_link
                || null;
            if (!link) continue;
            const active = ad.effective_status === 'ACTIVE';
            if (!chosen[cid] || (active && !chosen[cid].active)) chosen[cid] = { li
        }
        for (const cid in chosen) byId[cid].link = chosen[cid].link;
    } catch (e) { /* bỏ qua link nếu lỗi */ }
    })(),

// (3) Nhóm quảng cáo (ad set): ngân sách + chi tiêu từng nhóm
(async () => {
    try {
        const [adsets, asInsights] = await Promise.all([
            fbAll(`act_${acc}/adsets`, {
                fields: 'id,name,campaign_id,daily_budget,lifetime_budget,effective_s
                limit: 500,
            }, token),

```

```

        fbAll(`act_${acc}/insights`, {
            level: 'adset',
            fields: 'adset_id,spend,actions',
            time_range: { since, until },
            limit: 1000,
        }, token),
    ]);
    const asById = {};
    for (const a of adsets) {
        if (!byId[a.campaign_id]) continue;
        asById[a.id] = {
            campaign_id: a.campaign_id,
            name: a.name,
            budget: Number(a.daily_budget || a.lifetime_budget || 0),
            on: a.effective_status === 'ACTIVE',
            spent: 0, results: 0,
        };
    }
    for (const row of asInsights) {
        const as = asById[row.adset_id];
        if (!as) continue;
        const objective = byId[as.campaign_id] ? byId[as.campaign_id].objective
        as.spent = Number(row.spend || 0);
        as.results = countResults(row.actions, objective).value;
    }
    const budgetSum = {};
    for (const id in asById) {
        const as = asById[id];
        const camp = byId[as.campaign_id];
        if (!camp) continue;
        camp.adsets.push({ name: as.name, budget: as.budget, spent: as.spent, r
        if (as.on) budgetSum[as.campaign_id] = (budgetSum[as.campaign_id] || 0)
    }
    for (const cid in budgetSum) {
        if (byId[cid] && !byId[cid].budget) byId[cid].budget = budgetSum[cid];
    }
    } catch (e) { /* bỏ qua nhóm nếu lỗi */ }
    })(),

    ]);

    return Object.values(byId);
}

/* ----- Đăng nhập ----- */
app.set('trust proxy', 1);
// Lưu phiên đăng nhập ra FILE để KHÔNG bị đăng xuất khi máy chủ khởi động lại

```

```

const SESS_FILE = path.join(__dirname, 'sessions.json');
class FileSessionStore extends session.Store {
  constructor() {
    super();
    this.sessions = {};
    try { this.sessions = JSON.parse(fs.readFileSync(SESS_FILE, 'utf8')); } catch
  }
  _save() { try { fs.writeFileSync(SESS_FILE, JSON.stringify(this.sessions)); } c
  get(sid, cb) { try { const s = this.sessions[sid]; cb(null, s ? JSON.parse(s) :
  set(sid, sess, cb) { this.sessions[sid] = JSON.stringify(sess); this._save(); i
  destroy(sid, cb) { delete this.sessions[sid]; this._save(); if (cb) cb(null); }
  touch(sid, sess, cb) { this.sessions[sid] = JSON.stringify(sess); this._save();
}

app.use(session({
  store: new FileSessionStore(),
  secret: process.env.SESSION_SECRET || 'doi-thanh-mot-chuoi-bi-mat-ngau-nhien',
  resave: false,
  saveUninitialized: false,
  // KHÔNG đặt maxAge -> cookie phiên: chỉ đăng xuất khi ĐÓNG trình duyệt,
  // và vì phiên lưu ra file nên Redeploy/Restart không làm đăng xuất nữa.
  cookie: { httpOnly: true, sameSite: 'lax', maxAge: 30 * 24 * 60 * 60 * 1000 },
}));
app.use(express.urlencoded({ extended: false }));
app.use(express.json());

function loginPage(error) {
  return `<!DOCTYPE html><html lang="vi"><head><meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0"><title>Đăng
<style>
  body{margin:0;font-family:system-ui,sans-serif;background:#10192B;color:#fff;di
  .box{background:#1C2A42;border:1px solid rgba(255,255,255,.08);border-radius:16
  h1{font-size:18px;margin:0 0 4px;} p{color:#9FB0C8;font-size:13px;margin:0 0 20
  label{display:block;font-size:13px;color:#C4D0E2;margin:14px 0 6px;}
  input{width:100%;box-sizing:border-box;padding:11px 12px;border-radius:9px;bord
  input:focus{border-color:#0E8C76;}
  button{width:100%;margin-top:20px;padding:12px;border:none;border-radius:9px;ba
  .err{background:#3a1d1d;color:#ffb4a8;font-size:13px;padding:9px 11px;border-ra
</style></head><body>
<form class="box" method="POST" action="/login">
  <h1>Check ADS TDMJSC</h1><p>Đăng nhập để xem báo cáo</p>
  <label>Tên đăng nhập</label><input name="user" autocomplete="username" required
  <label>Mật khẩu</label><input name="pass" type="password" autocomplete="current
  ${error ? '<div class="err">Sai tên đăng nhập hoặc mật khẩu.</div>' : ''}
  <button type="submit">Đăng nhập</button>
</form></body></html>`;
}

```



```

app.get('/login', (req, res) => res.send(loginPage(req.query.error)));
app.post('/login', (req, res) => {
  const { user, pass } = req.body;
  const u = USERS.find(x => x.user === user && x.pass === pass);
  if (!u) return res.redirect('/login?error=1');
  req.session.user = { user: u.user, role: u.role, employees: u.employees || [],
  res.redirect(u.role === 'product' ? '/products.html' : (u.role === 'staff' ? '/'
});
app.get('/logout', (req, res) => req.session.destroy(() => res.redirect('/login'))

// =====
// GẮN MODULE THÁI LAN (an toàn – lỗi ở đây KHÔNG làm sập app chính)
// Đặt TRƯỚC middleware bắt buộc đăng nhập để webhook Ladipage gọi được.
// Import động mysql2 + module thailand, bọc try/catch toàn bộ.
// =====
(async () => {
  try {
    const [{ mountThailand }, mysqlMod] = await Promise.all([
      import('./thailand.js'),
      import('mysql2/promise'),
    ]);
    const mysql = mysqlMod.default || mysqlMod;
    mountThailand(app, { mysql, express, getCampaigns, QC_TAX: Number(process.env
      loadOwners, normProd, detectEmployee,
      exposeCounter: (fn) => { global.__thaiOrderCounts = fn; } });
    //Expose cho Thailand salary sync
    global.__SALARY_MANUAL = SALARY_MANUAL;
    global.__saveManual = saveManual;
  } catch (e) {
    console.error('[thailand] KHÔNG gắn được module (app chính vẫn chạy bình thườ
  }
})();

// =====
// Import động module OMS Việt Nam
// =====
(async () => {
  try {
    const [{ mountOMS }, mysqlMod] = await Promise.all([
      import('./oms.js'),
      import('mysql2/promise'),
    ]);
    const mysql = mysqlMod.default || mysqlMod;
    mountOMS(app, { mysql, express });
  } catch (e) {
    console.error('[OMS] KHÔNG gắn được module (app chính vẫn chạy bình thường):'

```

```

    }
  })();

// Từ đây trở xuống yêu cầu đăng nhập
app.use((req, res, next) => {
  // BỎ QUA mọi đường dẫn /thailand - module Thailand TỰ LO auth riêng (webhook +
  // (mountThailand chạy async nên route /thailand đăng ký SAU middleware này → p
  if (req.path === '/thailand' || req.path.startsWith('/thailand/')) return next(
  // BỎ QUA mọi đường dẫn /oms - module OMS TỰ LO auth riêng
  if (req.path === '/oms' || req.path.startsWith('/oms/') || req.path === '/go-om
    || req.path === '/oms-admin.html' || req.path === '/sw-push.js'
    || req.path === '/oms-manifest.json' || req.path === '/api/oms-button') retur
  if (req.session && req.session.user) return next();
  if (req.path.startsWith('/api/')) return res.status(401).json({ error: 'Chưa đã
  res.redirect('/login');
});

// Cho trang biết người đang đăng nhập là ai (để ẩn/hiện menu, lọc theo người)
app.get('/api/me', (req, res) => {
  const u = req.session.user || {};
  const displayName = u.salaryName || u.manager || (u.employees && u.employees[0]
  res.json({ user: u.user, role: u.role, manager: u.manager || '', employees: u.e
});

// QUYỀN "product": chỉ được vào trang Sản phẩm + API sản phẩm của mình.
// Chặn Dashboard, Marketing và mọi API khác.
app.use((req, res, next) => {
  const me = req.session.user;
  if (me && me.role === 'product') {
    const p = req.path;
    const allowed = ['/products.html', '/my-salary.html', '/logout', '/api/produc
    if (!allowed) {
      if (p.startsWith('/api/')) return res.status(403).json({ error: 'Không có q
      return res.redirect('/products.html');
    }
  }
});

// QUYỀN "staff": chỉ xem lương của mình (my-salary.html)
if (me && me.role === 'staff') {
  const p = req.path;
  const allowed = ['/my-salary.html', '/logout', '/api/me', '/api/my-salary/mon
  if (!allowed) {
    if (p.startsWith('/api/')) return res.status(403).json({ error: 'Không có q
    return res.redirect('/my-salary.html');
  }
}

// Marketing (viewer): chỉ Dashboard + Marketing, KHÔNG vào trang Sản phẩm

```

```

if (me && me.role === 'viewer') {
  const p = req.path;
  if (p === '/products.html' || p.startsWith('/api/products')) {
    if (p.startsWith('/api/')) return res.status(403).json({ error: 'Không có q
    return res.redirect('/');
  }
}
// — CHỈ ADMIN: trang Lương + Kết quả kinh doanh —
// Liệt kê tường minh để không sót khi thêm trang mới.
if (me && me.role !== 'admin') {
  const p = req.path;
  const TRANG_CHI_ADMIN = [
    '/salary.html',          // Lương Marketing
    '/salary-product.html',  // Lương Phát triển sản phẩm
    '/ket-qua-kinh-doanh.html', // Kết quả kinh doanh
  ];
  const API_CHI_ADMIN =
    p.startsWith('/api/salary')          // gồm cả /api/salary-product/*
    || p.startsWith('/api/business-result') // Kết quả kinh doanh
    || p.startsWith('/api/bank-accounts')  // STK nhân viên (chuyển lương)
    || p.startsWith('/api/meta-cache')     // ghim / xoá / nạp lại cache
    || p.startsWith('/api/admin/');
  if (TRANG_CHI_ADMIN.includes(p) || API_CHI_ADMIN) {
    if (p.startsWith('/api/')) return res.status(403).json({ error: 'Chỉ quản t
    return res.redirect('/');
  }
}
next();
});

// Bộ nhớ đệm tạm trong RAM: giữ kết quả theo từng khoảng ngày trong vài phút,
// để các lần mở/đăng nhập lại không phải gọi lại Meta -> nhanh hơn nhiều.
const DATA_CACHE = new Map();
const CACHE_MS = 3 * 60 * 1000; // 3 phút

// ===== CACHE VĨNH VIỄN cho các khoảng ngày ĐÃ QUA (chi tiêu Meta cố định) =====
// Lưu file trong DATA_DIR. Chỉ cache khi until < hôm nay (dữ liệu không còn đổi)
const META_CACHE_FILE = path.join(DATA_DIR, 'meta-cache.json');
let META_CACHE = {};
try { META_CACHE = JSON.parse(fs.readFileSync(META_CACHE_FILE, 'utf8')); } catch
let metaCacheSaveTimer = null;
function saveMetaCache() {
  // Gộp nhiều lần ghi trong 2 giây thành 1 lần để đỡ ghi đĩa liên tục
  clearTimeout(metaCacheSaveTimer);
  metaCacheSaveTimer = setTimeout(() => {
    try { fs.writeFileSync(META_CACHE_FILE, JSON.stringify(META_CACHE)); } catch
  }, 2000);

```

```

}
// Khoảng [since, until] đã hoàn toàn nằm trong quá khứ chưa? (until < hôm nay th
function isPastRange(until) {
  const today = new Date();
  // Giờ VN = UTC+7
  const vnNow = new Date(today.getTime() + 7 * 3600 * 1000);
  const todayStr = vnNow.toISOString().slice(0, 10);
  return until < todayStr; // until trước hôm nay → cố định
}

// Nạp 1 tài khoản, tự thử lại nếu lỗi (token nặng/timeout thỉnh thoảng rớt)
async function fetchAccountRetry(acc, token, days, since, until, tries = 3) {
  let lastErr;
  for (let i = 0; i < tries; i++) {
    try { return await fetchAccount(acc, token, days, since, until); }
    catch (e) { lastErr = e; await sleep(800); }
  }
  throw lastErr;
}

// Nạp toàn bộ campaign (mọi tài khoản). CHỈ lưu đệm khi nạp ĐỦ tất cả tài khoản,
// để tránh lưu nhầm dữ liệu thiếu (gây dashboard/lương về 0).
async function getCampaigns(since, until, opts = {}) {
  const days = listDays(since, until);
  const key = since + '|' + until;
  const past = isPastRange(until);
  const force = !!opts.force; // ép nạp lại từ Facebook, bỏ qua cache thường

  // (0) CACHE ĐÃ GHIM: số liệu do người dùng XÁC NHẬN ĐÚNG → luôn dùng, kể cả kh
  // Chỉ khi bỏ ghim (unpin) mới nạp lại được. Đây là lớp bảo vệ số liệu đã c
  const pinnedRec = META_CACHE[key];
  if (pinnedRec && pinnedRec.pinned && Array.isArray(pinnedRec.campaigns)) {
    return pinnedRec.campaigns;
  }

  // (1) Khoảng đã qua: nếu có trong cache file → trả luôn, KHÔNG gọi API Faceboo
  if (!force && past && META_CACHE[key] && Array.isArray(META_CACHE[key].campaign
    return META_CACHE[key].campaigns;
  }

  // (2) Cache RAM ngắn hạn (3 phút) cho khoảng có hôm nay
  if (!force) {
    const cached = DATA_CACHE.get(key);
    if (cached && cached.complete && Date.now() - cached.at < CACHE_MS) return ca
  }

  // (3) Gọi API Facebook – theo dõi TỪNG tài khoản
  const accList = [];

```

```

for (const src of SOURCES)
  for (const acc of src.accounts)
    accList.push({ acc, token: src.token });

const tasks = accList.map(a => fetchAccountRetry(a.acc, a.token, days, since, u
const results = await Promise.allSettled(tasks);
const campaigns = [];
let failed = 0;
const emptyAccts = []; // tài khoản trả về THÀNH CÔNG nhưng RỖNG
results.forEach((r, i) => {
  if (r.status === 'fulfilled') {
    const arr = r.value || [];
    campaigns.push(...arr);
    if (arr.length === 0) emptyAccts.push(accList[i].acc);
  } else {
    failed++;
  }
});

// "Đủ dữ liệu" = không tài khoản nào LỖI và có ít nhất 1 campaign.
// (Tài khoản rỗng là BÌNH THƯỜNG – nhiều tài khoản không chạy QC trong khoảng
// Trước đây chặn cache khi có bất kỳ tài khoản rỗng nào → cache gần như KHÔNG
// khiến mỗi lần vào phải gọi lại Facebook và thỉnh thoảng bị thiếu ngân sách.
const complete = (failed === 0);
const fullData = (failed === 0 && campaigns.length > 0);

if (complete) {
  DATA_CACHE.set(key, { at: Date.now(), campaigns, complete: true });
}
// Khoảng đã qua + lấy đủ → LƯU VĨNH VIỄN (không đung vào bản đã ghim)
if (past && fullData && !(META_CACHE[key] && META_CACHE[key].pinned)) {
  META_CACHE[key] = { at: new Date().toISOString(), campaigns };
  saveMetaCache();
  console.log(`[getCampaigns] Đã lưu cache vĩnh viễn ${key}: ${campaigns.length}`);
}
if (past && !fullData) {
  console.warn(`[getCampaigns] KHÔNG lưu cache vĩnh viễn cho ${key}: failed=${f
}
return campaigns;
}

// Xoá cache RAM (dùng chung cho các endpoint xoá cache)
function clearRamCache(keys) {
  if (!keys) { DATA_CACHE.clear(); return; }
  for (const k of keys) DATA_CACHE.delete(k);
}

```

```

// XÓA CACHE ngân sách Meta đã lưu (dùng khi cache bị lưu nhầm dữ liệu thiếu)
// - Xóa 1 khoảng: /api/admin/clear-cache?since=2026-06-11&until=2026-06-24
// - Xóa KHOẢNG GIAO nhau: /api/admin/clear-cache?overlaps=2026-06-11..2026-06-2
// - Xóa tất cả: /api/admin/clear-cache?all=1
app.get('/api/admin/clear-cache', (req, res) => {
  const me = req.session && req.session.user;
  if (!me || me.role !== 'admin') return res.status(403).json({ ok: false, messag

  const before = Object.keys(META_CACHE).length;
  const removed = [];
  const kept = []; // các khoảng ĐÃ GHIM → không xóa
  const isPinned = k => !! (META_CACHE[k] && META_CACHE[k].pinned);
  // ?includePinned=1 → xóa cả những khoảng đã ghim (dùng khi thật sự muốn làm lạ
  const includePinned = req.query.includePinned === '1';

  if (req.query.all === '1') {
    for (const k of Object.keys(META_CACHE)) {
      if (isPinned(k) && !includePinned) { kept.push(k); continue; }
      removed.push(k); delete META_CACHE[k];
    }
    clearRamCache(); // xóa SẠCH cache RAM, nếu không sẽ vẫn trả số cũ trong 3
  } else if (req.query.overlaps) {
    // overlaps=YYYY-MM-DD..YYYY-MM-DD : xóa mọi cache key mà khoảng của nó GIAO
    const [a, b] = String(req.query.overlaps).split('..');
    if (a && b) {
      for (const k of Object.keys(META_CACHE)) {
        const [ks, ku] = k.split('|');
        // giao nhau nếu ks <= b && ku >= a
        if (ks && ku && ks <= b && ku >= a) {
          if (isPinned(k) && !includePinned) { kept.push(k); continue; }
          removed.push(k); delete META_CACHE[k];
        }
      }
      // Xóa cả cache RAM của những khoảng giao nhau (kể cả khoảng chưa có trong
      for (const k of DATA_CACHE.keys()) {
        const [ks, ku] = k.split('|');
        if (ks && ku && ks <= b && ku >= a) DATA_CACHE.delete(k);
      }
    }
  } else if (req.query.since && req.query.until) {
    const key = req.query.since + '|' + req.query.until;
    if (isPinned(key) && !includePinned) {
      kept.push(key);
    } else if (META_CACHE[key]) {
      removed.push(key); delete META_CACHE[key];
    }
    DATA_CACHE.delete(key); // xóa cả RAM cache của khoảng này
  }
}

```

```

    } else {
      return res.json({ ok: false, message: 'Cần tham số: ?all=1 HOẶC ?since=..&unt
    }

// Lưu lại file cache sau khi xóa
try { fs.writeFileSync(META_CACHE_FILE, JSON.stringify(META_CACHE)); } catch (e
res.json({
  ok: true,
  đã_xóa: removed.length,
  giữ_lại_vì_đã_ghim: kept.length,
  còn_lại: Object.keys(META_CACHE).length,
  cac_key_da_xoa: removed,
  cac_key_da_ghim: kept,
});
});

/* — GHIM CACHE: số liệu đã XÁC NHẬN ĐÚNG, khoá lại vĩnh viễn —
Khoảng đã ghim: không bị xoá bởi nút xoá cache, không bị nạp lại bởi nút tải l
Muốn cập nhật thì phải BỎ GHIM trước.

// Ghim khoảng ngày hiện tại: POST /api/meta-cache/pin {since, until}
app.post('/api/meta-cache/pin', express.json(), async (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chi
  const { since, until } = req.body || {};
  if (!since || !until) return res.json({ ok: false, message: 'Cần since và until
  const key = since + '|' + until;
  try {
    // Ưu tiên dữ liệu đang có sẵn (chính là số bạn vừa nhìn thấy và xác nhận đún
    let campaigns = null;
    if (META_CACHE[key] && Array.isArray(META_CACHE[key].campaigns)) campaigns =
    else {
      const ram = DATA_CACHE.get(key);
      if (ram && Array.isArray(ram.campaigns)) campaigns = ram.campaigns;
    }
    // Chưa có gì trong cache → nạp từ Facebook rồi ghim
    if (!campaigns) campaigns = await getCampaigns(since, until);
    if (!campaigns.length)
      return res.json({ ok: false, message: 'Khoảng này không có chiến dịch nào —

    const budget = campaigns.reduce((s, c) => s + (Number(c.budget) || 0), 0);
    META_CACHE[key] = {
      at: new Date().toISOString(),
      campaigns,
      pinned: true,
      pinnedAt: new Date().toISOString(),
      pinnedBy: me.user || '',

```

```

    };
    saveMetaCache();
    res.json({
      ok: true, since, until,
      soChienDich: campaigns.length,
      tongNganSach: budget,
      message: `Đã ghim ${campaigns.length} chiến dịch cho ${since} → ${until}. S
    });
  } catch (e) {
    res.json({ ok: false, message: e.message });
  }
});

// Bỏ ghim: POST /api/meta-cache/unpin {since, until, xoaLuon}
app.post('/api/meta-cache/unpin', express.json(), (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chỉ
  const { since, until, xoaLuon } = req.body || {};
  if (!since || !until) return res.json({ ok: false, message: 'Cần since và until
  const key = since + '|' + until;
  if (!META_CACHE[key]) return res.json({ ok: false, message: 'Khoảng này chưa có
  if (!META_CACHE[key].pinned) return res.json({ ok: false, message: 'Khoảng này

  if (xoaLuon) {
    delete META_CACHE[key];          // bỏ ghim + xoá luôn → lần sau nạp mới từ Faceb
    DATA_CACHE.delete(key);
  } else {
    delete META_CACHE[key].pinned;
    delete META_CACHE[key].pinnedAt;
    delete META_CACHE[key].pinnedBy;
  }
  saveMetaCache();
  res.json({ ok: true, message: xoaLuon ? 'Đã bỏ ghim và xoá cache – lần sau sẽ n
});

app.get('/api/data', async (req, res) => {
  try {
    const since = req.query.since || '2026-06-01';
    const until = req.query.until || '2026-06-09';
    const days = listDays(since, until);
    // ?refresh=1 → bỏ qua cache, nạp lại từ Facebook (nút "Tải lại" ở dashboard)
    const force = req.query.refresh === '1' || req.query.force === '1';
    const campaigns = await getCampaigns(since, until, { force });

    // Lọc theo quyền: viewer chỉ thấy nhân viên được phép
    const me = req.session.user;
    let visible = campaigns;

```



```

    if (me.role !== 'admin') {
        const allow = new Set(me.employees || []);
        visible = campaigns.filter(c => allow.has(c.employee));
    }
    res.json({ days, campaigns: visible, me: { user: me.user, role: me.role, disp
} catch (e) {
    res.status(500).json({ error: e.message });
}
});

/* =====
MARKETING – lấy từ API Sandbox (trang quản lý đơn hàng)
Xem ĐƠN / DOANH THU theo từng nhân viên marketing.
- Cần biến môi trường SANDBOX_TOKEN (sinh trong trang quản trị Sandbox:
  {domain}/caidat/he-thong → tab "Cài đặt API truy vấn data").
- API giới hạn 60 GIÂY/LẦN mỗi endpoint → phải tải NGẦM, có bộ nhớ đệm.
- Mỗi request lọc tối đa 1 THÁNG, mỗi trang tối đa 100 bản ghi.
===== */
const SANDBOX_BASE = process.env.SANDBOX_BASE || 'https://api.sandbox.com.vn/part
const SANDBOX_TOKEN = process.env.SANDBOX_TOKEN || '';
// Chi nhánh: mặc định null = (chi nhánh duy nhất của bạn). Có thể khai
// SANDBOX_BRANCH = '<mã chi nhánh>' nếu sau này cần lọc riêng.
const _branchEnv = process.env.SANDBOX_BRANCH;
const SANDBOX_BRANCH = (_branchEnv && _branchEnv !== 'null') ? _branchEnv : null;
const MKT_PAGE_DELAY_MS = 62 * 1000; // chờ 62s giữa các trang (API giới hạn ~60s
const MKT_MAX_PAGES = 40; // tối đa 40 trang/lần (mỗi trang ~60s)

const sleep = ms => new Promise(r => setTimeout(r, ms));
// Cộng 1 ngày (YYYY-MM-DD) -> dùng cho denNgay để lấy trọn ngày cuối
const addDay = s => { const d = new Date(s + 'T00:00:00Z'); d.setUTCDate(d.getUTC

// Trạng thái + kết quả marketing (giữ trong RAM, không chặn request)
const MKT = { fetching: false, lastUpdated: null, since: null, until: null,
    rows: [], totalRecord: 0, loaded: 0, error: null };

// Đơn "đã giao thành công" – dựa vào MÃ trạng thái giao vận (orderStatusId).
// Theo dữ liệu thật: 31 = Đã giao hàng, 32 = Đã giao + đã đối soát.
// Nếu muốn tính thêm/bớt trạng thái, sửa danh sách SHIP_OK_STATUS dưới đây.
const SHIP_OK_STATUS = ['31', '32'];
function isShippedOK(o) {
    return SHIP_OK_STATUS.includes(String(o.orderStatusId));
}

/* ===== HƯỚNG A =====
Đăng nhập web Sandbox bằng tài khoản (lưu ở biến môi trường) để gọi
API BÁO CÁO "lead theo nhân sự" – chính xác theo NGÀY TẠO, 1 lần gọi.
Cần khai trên máy chủ:

```

```

    SANDBOX_WEB_USER = <tài khoản web>
    SANDBOX_WEB_PASS = <mật khẩu web>
    (Mã chi nhánh đã điền sẵn; đổi qua SANDBOX_CHINHANH nếu cần.)
    ===== */
const SANDBOX_WEB_USER = process.env.SANDBOX_WEB_USER || '';
const SANDBOX_WEB_PASS = process.env.SANDBOX_WEB_PASS || '';
const SANDBOX_AUTH_BASE = process.env.SANDBOX_AUTH_BASE || 'https://api.sandbox.c
const SANDBOX_REPORT_URL = process.env.SANDBOX_REPORT_URL
    || 'https://api.sandbox.com.vn/report/api/report/ReportLeadByNhanSuMktSearch';
const SANDBOX_CHINHANH = process.env.SANDBOX_CHINHANH || 'f13cc0dc-28f0-4d17-b7a
// Mã thiết bị (lấy từ cookie prodevice_id của trình duyệt bạn) - để server "giống
const SANDBOX_DEVICE_ID = process.env.SANDBOX_DEVICE_ID || '2bfd88c7-ea01-482e-8b
// Tên miền của bạn - server gửi kèm để Sandbox biết bạn thuộc tổ chức nào (như ô
const SANDBOX_ORIGIN = (process.env.SANDBOX_ORIGIN || 'https://tdmjsc.sandbox.com

let sandboxCookie = '';          // chuỗi Cookie sau khi đăng nhập web
let sandboxLoginAt = null;

// Mã hoá một chuỗi (userName hoặc password) y hệt web Sandbox:
// 1) sinh khoá 16 ký tự, 2) AES-128-ECB/Pkcs7 -> base64,
// 3) chèn khoá ngay sau CHỮ SỐ ĐẦU TIÊN trong chuỗi base64 (server tự tách ra).
function sandboxEncrypt(text) {
    const key = crypto.randomBytes(12).toString('base64').replace(/[+|=]/g, 'x').sl
    const cipher = crypto.createCipheriv('aes-128-ecb', Buffer.from(key, 'utf8'), n
    const b64 = cipher.update(text, 'utf8', 'base64') + cipher.final('base64');
    let f = -1;
    for (let i = 0; i < b64.length; i++) { const c = b64[i]; if (c >= '0' && c <= '
    if (f === -1) return key + b64; // không có chữ số -> theo đúng hành vi substri
    return b64.slice(0, f) + key + b64.slice(f);
}

// Đăng nhập web -> lấy cookie phiên. KHÔNG log mật khẩu.
async function sandboxLogin() {
    if (!SANDBOX_WEB_USER || !SANDBOX_WEB_PASS)
        throw new Error('Chưa khai SANDBOX_WEB_USER / SANDBOX_WEB_PASS trên máy chủ.')
    const r = await fetchWithTimeout(SANDBOX_AUTH_BASE + 'api/Authen/login-encrypt'
    method: 'POST',
    headers: {
        'Content-Type': 'application/json', 'Accept': 'application/json, text/plain
        'Origin': SANDBOX_ORIGIN, 'Referer': SANDBOX_ORIGIN + '/',
        'Cookie': `prodevice_id=${SANDBOX_DEVICE_ID}`,
    },
    body: JSON.stringify({
        userName: sandboxEncrypt(SANDBOX_WEB_USER),
        password: sandboxEncrypt(SANDBOX_WEB_PASS),
        deviceId: SANDBOX_DEVICE_ID,
        captcha: '',
    })
}

```

```

    }},
  });
  let setCookies = [];
  try { setCookies = r.headers.getSetCookie ? r.headers.getSetCookie() : []; } ca
  if (!setCookies || !setCookies.length) { const sc = r.headers.get('set-cookie')
  const jar = { prodevice_id: SANDBOX_DEVICE_ID };
  for (const c of setCookies) { const p = c.split(';')[0]; const i = p.indexOf('=
  sandboxCookie = Object.entries(jar).map(([k, v]) => `${k}=${v}`).join('; ');
  let body = null; try { body = await r.json(); } catch (e) {}
  sandboxLoginAt = new Date().toISOString();
  return {
    httpStatus: r.status, coGanCookie: Object.keys(jar).length > 1, soCookie: Obj
    success: body ? (body.success ?? body.Success) : undefined,
    message: body ? (body.message || body.Message) : undefined,
  };
}

```

```

// Gọi API báo cáo cho khoảng ngày [since, until] (YYYY-MM-DD). Tự đăng nhập lại
async function sandboxReport(since, until) {
  const tuNgay = `${since}T00:00:00.000+07:00`;
  const denNgay = `${until}T23:59:59.998+07:00`;
  const payload = {
    pageInfo: { page: 1, pageSize: 1000 }, sorts: [],
    kieuXem: 4, loaiNhanVien: 1, isChietKhau: true, isVat: true,
    date: [tuNgay, denNgay], tuNgay, denNgay,
    idChiNhanh: SANDBOX_CHINHANH, kieuNgay: 'NgayTao',
    typeViewDetail: null, idPhongBanSale: null, idNhomNhanVienSale: null, idUsers
    idPhongBanMkts: null, idNhomNhanVienMkts: null, idUserMkts: null,
  };
  const call = () => fetch(SANDBOX_REPORT_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json', 'Accept': 'application/json, text/plain
      'Origin': SANDBOX_ORIGIN, 'Referer': SANDBOX_ORIGIN + '/',
      'Cookie': sandboxCookie,
    },
    body: JSON.stringify(payload),
  });
  if (!sandboxCookie) await sandboxLogin();
  let r = await call();
  if (r.status === 401 || r.status === 403) { await sandboxLogin(); r = await cal
  const j = await r.json().catch(() => ({ success: false, message: 'Phản hồi khôn
  return { httpStatus: r.status, json: j };
}

```

```

// Chuyển dữ liệu báo cáo -> dạng bảng cho dashboard
function mapReport(j) {

```

```

const d = (j && j.data) || {};
const rows = (d.reportLeadByNhanSuMktDtos || []).map(r => ({
  name: (r.ten || '').trim(), contact: r.soContact, chot: r.soDonChot,
  tyLe: r.tyLeChotDon, soSP: r.soLuongSanPham, doanhthu: r.doanhSo,
  nganSach: r.nganSach, giaContact: r.giaContact,
}));
const t = d.reportLeadByNhanSuMktTotalDto || {};
const total = {
  contact: t.tongSoContact, chot: t.tongSoDonHang, soSP: t.tongSanPham,
  tyLe: t.tongTyLeChot, doanhthu: t.tongDoanhSo,
  nganSach: t.tongNganSach, giaContact: t.giaContact,
};
return { rows, total };
}

function aggregateMarketing(orders, acc) {
  for (const o of orders) {
    const key = ((o.marketingDisplayName || o.marketingUserName || '').trim()) ||
    if (!acc[key]) acc[key] = { name: key, data: 0, chot: 0, ship: 0, doanhthu: 0
    const a = acc[key];
    a.data += 1; // số đơn
    if (String(o.orderConfirmId) === '1') { // 1 = đã chốt đơn
      a.chot += 1; // đơn chốt thành công
      a.doanhthu += Number(o.totalPrice || 0); // tiền đơn cuối
    }
    if (isShippedOK(o)) a.ship += 1; // đơn ship thành công
    // Gom sản phẩm của đơn (cần isIncludeDetail = true)
    const items = Array.isArray(o.details) ? o.details : [];
    const seen = new Set();
    for (const d of items) {
      const ten = ((d.itemName || d.tenSanPham || d.productName || d.name || ''
      if (!ten) continue;
      if (!a.products[ten]) a.products[ten] = { ten, soDon: 0, soLuong: 0 };
      a.products[ten].soLuong += Number(d.quantity || d.soLuong || 0);
      if (!seen.has(ten)) { a.products[ten].soDon += 1; seen.add(ten); } // số đo
    }
  }
}

async function refreshMarketing(since, until) {
  if (MKT.fetching) return;
  if (!SANDBOX_TOKEN) { MKT.error = 'Chưa khai SANDBOX_TOKEN trên máy chủ.'; retu
  MKT.fetching = true; MKT.error = null; MKT.since = since; MKT.until = until;
  MKT.rows = []; MKT.loaded = 0; MKT.totalRecord = 0;
  const acc = {};
  const seenOrders = new Set(); // mã đơn đã tính -> chống đếm trùng khi API trả
  let rateRetries = 0;

```

```

try {
  for (let page = 1; page <= MKT_MAX_PAGES; page++) {
    const res = await fetch(`${SANDBOX_BASE}/DonHangLogistic/GetOrderByConditio
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer $
      body: JSON.stringify({
        idChiNhanh: SANDBOX_BRANCH,
        kieuNgay: 'NgayTao',
        tuNgay: since,           // ngày-tron YYYY-MM-DD (API chỉ lọc đúng với
        denNgay: addDay(until),  // +1 ngày để lấy tron ngày cuối
        pageInfo: { page, pageSize: 100 },
        sorts: [],
        isIncludeDetail: true,  // lấy danh sách sản phẩm của đơn
        isHistories: false,
      })),
    });
    const json = await res.json().catch(() => ({ success: false, message: 'Phản
    if (!json.success) {
      const msg = String(json.message || json.Message || '');
      // Bị giới hạn tốc độ ("cần chờ Ns ...") -> đợi đúng số giây báo rồi thử
      if (/chờ|cần chờ|call api|giây|rate|quá nhanh/i.test(msg) && rateRetries
        const m = msg.match(/(\d+)\s*s/);
        const waitMs = (m ? Number(m[1]) + 3 : 62) * 1000;
        rateRetries++; await sleep(waitMs); page--; continue;
      }
      MKT.error = msg || 'API Sandbox báo lỗi (kiểm tra token).'; break;
    }
    rateRetries = 0;
    const orders = json.data || [];
    // Lọc: chỉ đơn MỚI (chống trùng) VÀ đúng khoảng ngày theo createTime (chốt
    let newSeen = 0;
    const fresh = [];
    for (const o of orders) {
      const id = String(o.orderId || o.orderNumber || o.orderCode || '');
      if (!id || seenOrders.has(id)) continue;
      seenOrders.add(id); newSeen++;
      const d = (o.createTime || '').slice(0, 10);
      if (d && (d < since || d > until)) continue; // ngoài khoảng ngày -> khôn
      fresh.push(o);
    }
    aggregateMarketing(fresh, acc);
    MKT.loaded = seenOrders.size;
    MKT.totalRecord = seenOrders.size;
    MKT.rows = Object.values(acc).map(r => {
      const prods = Object.values(r.products).sort((x, y) => y.soDon - x.soDon)
      return {
        name: r.name, soDon: r.data, chot: r.chot, ship: r.ship,

```

```

        soSP: prods.reduce((s, p) => s + p.soLuong, 0), // tổng số lượng sản ph
        doanhthu: r.doanhthu, products: prods,
    };
    }).sort((a, b) => (b.chot - a.chot) || (b.doanhthu - a.doanhthu)); // điền
    if (orders.length < 100) break; // trang chưa đủ 100 -> server đã hết dữ
    if (newSeen === 0) break; // không còn đơn mới -> dừng (tránh lặp v
    await sleep(MKT_PAGE_DELAY_MS); // tôn trọng giới hạn tốc độ API (~60s)
    }
    MKT.lastUpdated = new Date().toISOString();
} catch (e) {
    MKT.error = e.message;
} finally {
    MKT.fetching = false;
}
}

app.get('/api/marketing', (req, res) => {
    res.json({
        ver: 'mkt-2026-06-15-v11', // hướng A: đăng nhập web + API báo cáo
        fetching: MKT.fetching, lastUpdated: MKT.lastUpdated,
        since: MKT.since, until: MKT.until,
        rows: MKT.rows, totalRecord: MKT.totalRecord, loaded: MKT.loaded,
        error: MKT.error, hasToken: !!SANDBOX_TOKEN,
    });
});

app.post('/api/marketing/refresh', (req, res) => {
    const today = new Date().toISOString().slice(0, 10);
    const since = (req.body && req.body.since) || (today.slice(0, 8) + '01');
    const until = (req.body && req.body.until) || today;
    if (MKT.fetching) return res.json({ started: false, message: 'Đang cập nhật, vu
    refreshMarketing(since, until); // chạy NGẦM, không await
    res.json({ started: true, since, until });
});

// (HƯỚNG A) Kiểm tra đăng nhập web + thủ lấy báo cáo 1 ngày. KHÔNG lộ mật khẩu.
// Mở: /api/marketing/login-test?day=2026-06-12
app.get('/api/marketing/login-test', async (req, res) => {
    const day = req.query.day || new Date(Date.now() - 864e5).toISOString().slice(0
    if (!SANDBOX_WEB_USER || !SANDBOX_WEB_PASS)
        return res.json({ ok: false, buoc: 'cauHinh', message: 'Chưa khai SANDBOX_WEB
    try {
        const login = await sandboxLogin();
        let report = null, total = null;
        if (login.coGanCookie) {
            const rp = await sandboxReport(day, day);
            report = { httpStatus: rp.httpStatus, success: rp.json && (rp.json.success

```

```

        const m = mapReport(rp.json); total = m.total; report.soNhanVien = m.rows.1
    }
    res.json({ ok: !(login.coGanCookie && total && total.contact !== null), day,
    } catch (e) {
        res.json({ ok: false, message: e.message });
    }
});

```

```

// (HƯỚNG A) Lấy báo cáo lead theo nhân sự cho khoảng ngày (mặc định hôm qua).
// Gộp chỉ tiêu thực từ Meta API vào cột "chiTieu" và tính lại "giaContact".
// Mở: /api/marketing/report?since=2026-06-12&until=2026-06-12
app.get('/api/marketing/report', async (req, res) => {
    const today = new Date().toISOString().slice(0, 10);
    const since = req.query.since || today;
    const until = req.query.until || since;
    try {
        // Chạy song song: Sandbox report + Meta spend + đơn Thái
        const [rp, metaSpend, thaiCounts] = await Promise.all([
            sandboxReport(since, until),
            (async () => {
                try {
                    // Tổng hợp chỉ tiêu Meta theo nhân viên (dùng lại getCampaigns đã có)
                    const campaigns = await getCampaigns(since, until);
                    const norm = s => String(s == null ? '' : s).trim().toLowerCase().replace
                    const spend = {};
                    for (const c of campaigns) {
                        const emp = norm(c.employee || '');
                        if (!emp || emp === 'chưa xác định') continue;
                        const s = (c.daily || []).reduce((t, d) => t + (Number(d.spent) || 0)
                        spend[emp] = (spend[emp] || 0) + s;
                    }
                    return spend;
                } catch (e) {
                    return {}; // nếu Meta lỗi vẫn trả Sandbox bình thường
                }
            })(),
            (async () => {
                try { return (typeof global.__thaiOrderCounts === 'function') ? await glo
                catch (e) { return {}; }
            })(),
        ]);
    }
});

```

```

if (!(rp.json && (rp.json.success ?? rp.json.Success)))
    return res.json({ ok: false, since, until, httpStatus: rp.httpStatus, messa

```

```

const m = mapReport(rp.json);
const norm = s => String(s == null ? '' : s).trim().toLowerCase().replace(/\s

```

```

// Gắn chỉ tiêu Meta vào từng dòng, tính lại giaContact
let rows = m.rows.map(r => {
  // Dùng chung QC_TAX với module lương (mặc định 11%, cấu hình qua .env)
  const TAX = 1 + QC_TAX;
  const chiTieu = Math.round((metaSpend[norm(r.name)] || 0) * TAX);
  const giaContact = (chiTieu > 0 && r.contact > 0) ? Math.round(chiTieu / r.
  const donThai = thaiCounts[norm(r.name)] || 0;
  const tongDon = (Number(r.contact) || 0) + donThai; // TỔNG ĐƠN = số conta
  const cpa = (chiTieu > 0 && tongDon > 0) ? Math.round(chiTieu / tongDon) :
  return { ...r, chiTieu, giaContact, donThai, tongDon, cpa };
});

// Lọc theo quyền
const me = req.session.user || {};
if (me.role !== 'admin') {
  const allow = new Set((me.employees || []).map(norm));
  rows = rows.filter(r => allow.has(norm(r.name)));
}

// Tính lại Tổng
const s = rows.reduce((a, r) => ({
  contact: a.contact + (+r.contact || 0),
  chot: a.chot + (+r.chot || 0),
  soSP: a.soSP + (+r.soSP || 0),
  doanhthu: a.doanhthu + (+r.doanhthu || 0),
  chiTieu: a.chiTieu + (+r.chiTieu || 0),
  donThai: a.donThai + (+r.donThai || 0),
}), { contact: 0, chot: 0, soSP: 0, doanhthu: 0, chiTieu: 0, donThai: 0 });

const total = {
  contact: s.contact,
  chot: s.chot,
  soSP: s.soSP,
  doanhthu: s.doanhthu,
  chiTieu: s.chiTieu,
  donThai: s.donThai,
  tongDon: s.contact + s.donThai,
  tyLe: s.contact ? (s.chot / s.contact * 100) : 0,
  giaContact: s.contact ? Math.round(s.chiTieu / s.contact) : 0,
  cpa: (s.contact + s.donThai) ? Math.round(s.chiTieu / (s.contact + s
});

rows.sort((a, b) => (b.doanhthu || 0) - (a.doanhthu || 0));
res.json({ ok: true, ver: 'mkt-2026-06-23-v13', since, until, rows, total, la
} catch (e) {
  res.json({ ok: false, since, until, message: e.message });
}

```



```
}  
});
```

```
// (TẠM – để gỡ lỗi) Xem cấu trúc dữ liệu thật từ Sandbox: mở /api/marketing/samp  
// để biết đúng tên trường sản phẩm + trạng thái giao hàng. Xóa sau khi đã chỉnh  
app.get('/api/marketing/sample', async (req, res) => {  
  if (!SANDBOX_TOKEN) return res.json({ error: 'Chưa khai SANDBOX_TOKEN' });  
  const today = new Date().toISOString().slice(0, 10);  
  const since = req.query.since || (today.slice(0, 8) + '01');  
  const until = req.query.until || today;  
  // Thử các kiểu gửi ngày khác nhau để tìm đúng định dạng API chấp nhận:  
  // ?fmt=iso (mặc định) | dateonly | dateplus | notz | utc ; ?kieu=NgàyTao (mặ  
  const fmt = req.query.fmt || 'iso';  
  const kieuNgay = req.query.kieu || 'NgàyTao';  
  const pi = Number(req.query.pi) || 1;  
  const addDay = s => { const d = new Date(s + 'T00:00:00Z'); d.setUTCDate(d.getU  
  const tuNgay = (fmt === 'dateonly' || fmt === 'dateplus') ? since  
    : fmt === 'notz' ? `${since}T00:00:00`  
    : fmt === 'utc' ? `${since}T00:00:00.000Z`  
    : `${since}T00:00:00+07:00`;  
  const denNgay = fmt === 'dateplus' ? addDay(until)  
    : fmt === 'dateonly' ? until  
    : fmt === 'notz' ? `${until}T23:59:59`  
    : fmt === 'utc' ? `${until}T23:59:59.999Z`  
    : `${until}T23:59:59+07:00`;  
  try {  
    const r = await fetch(`${SANDBOX_BASE}/DonHangLogistic/GetOrderByConditions`,  
      method: 'POST',  
      headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${S  
      body: JSON.stringify({  
        idChiNhanh: SANDBOX_BRANCH, kieuNgay,  
        tuNgay, denNgay,  
        pageInfo: { page: pi, pageSize: 100 }, sorts: [],  
        isIncludeDetail: true, isHistories: true,  
      })),  
    });  
    const json = await r.json();  
    const orders = json.data || [];  
    const o0 = orders[0] || {};  
    const statusRe = /status|giao|ship|deliver|trangthai|confirm/i;  
    const statusFields = {};  
    for (const k of Object.keys(o0)) if (statusRe.test(k)) statusFields[k] = o0[k]  
    const withDetails = orders.find(o => Array.isArray(o.details) && o.details.le  
    const dOnly = t => (t ? String(t).slice(0, 10) : null);  
    const createDates = orders.map(o => dOnly(o.createTime));  
    const outOfRange = createDates.filter(d => d && (d < since || d > until)).len  
    res.json({
```

```

    success: json.success, errMsg: json.message || json.Message || null,
    totalRecord: json.totalRecord, count: orders.length,
    triedFmt: fmt, triedKieu: kieuNgay, sentTuNgay: tuNgay, sentDenNgay: denNgay
    askedRange: [since, until],
    outOfRangeCount: outOfRange, // > 0 => bộ lọc ngày KHÔNG ăn
    distinctCreateDates: [...new Set(createDates)].sort(),
    createTimeSample: orders.slice(0, 20).map(o => o.createTime),
    dateFieldsSample: orders.slice(0, 12).map(o => ({
        create: o.createTime, recv: o.timeSaleReceivingData,
        confirm: o.orderConfirmDate, submit: o.timeOrderSubmit, update: o.updateT
    })),
    orderKeys: Object.keys(o0),
    statusFields,
    anyDetailsNonEmpty: orders.some(o => Array.isArray(o.details) && o.details.
    detailsSampleKeys: withDetails ? Object.keys(withDetails.details[0]) : null
    detailsSample: withDetails ? withDetails.details.slice(0, 2) : [],
    });
    } catch (e) { res.json({ error: e.message }); }
    });

```

```

// (TẠM – tự dò) Thử lần lượt các kiểu ngày để biết kiểu nào API lọc theo NGÀY TẠ
// Mở /api/marketing/kieu-test?day=2026-06-12 để BẮT ĐẦU (chạy ngầm ~5-6 phút),
// rồi mở lại chính link đó mỗi ~1 phút để xem kết quả điền dần. Xóa sau khi xong
const KIEU_TEST = { running: false, day: null, startedAt: null, finishedAt: null,
const KIEU_LIST = ['NgayTao', 'SaleNgayNhanData', 'DonHangNgayChot', 'NgayDangDon

```

```

async function runKieuTest(day) {
    KIEU_TEST.running = true; KIEU_TEST.day = day;
    KIEU_TEST.startedAt = new Date().toISOString(); KIEU_TEST.finishedAt = null;
    KIEU_TEST.results = []; KIEU_TEST.note = '';
    const tuNgay = `${day}T00:00:00+07:00`;
    const denNgay = `${day}T23:59:59+07:00`;
    const dOnly = t => (t ? String(t).slice(0, 10) : null);
    try {
        for (const kieu of KIEU_LIST) {
            let attempt = 0, done = false;
            while (!done && attempt < 6) {
                attempt++;
                const r = await fetch(`${SANDBOX_BASE}/DonHangLogistic/GetOrderByConditio
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer
                    body: JSON.stringify({
                        idChiNhanh: SANDBOX_BRANCH, kieuNgay: kieu,
                        tuNgay, denNgay, pageInfo: { page: 1, pageSize: 100 }, sorts: [],
                        isIncludeDetail: false, isHistories: false,
                    }),
            });
        }
    }
}

```

```

const json = await r.json().catch(() => ({ success: false, message: 'Phản
if (!json.success) {
  const msg = String(json.message || json.Message || '');
  if (/chờ|cần chờ|call api|giây|rate|quá nhanh/i.test(msg) && attempt <
    const m = msg.match(/(\d+)\s*s/);
    await sleep(((m ? Number(m[1]) : 60) + 3) * 1000); continue;
  }
  KIEU_TEST.results.push({ kieu, error: msg || 'lỗi', count: 0 });
  done = true; break;
}
const orders = json.data || [];
const inRange = field => orders.filter(o => dOnly(o[field]) === day).leng
KIEU_TEST.results.push({
  kieu, count: orders.length,
  createOnDay: inRange('createTime'),          // ngày tạo = đúng ngày?
  recvOnDay: inRange('timeSaleReceivingData'),
  confirmOnDay: inRange('orderConfirmDate'),
  submitOnDay: inRange('timeOrderSubmit'),
  updateOnDay: inRange('updateTime'),
  createSample: orders.slice(0, 3).map(o => o.createTime),
});
done = true;
}
await sleep(63 * 1000); // tôn trọng giới hạn ~60s/lần gọi
}
} catch (e) {
  KIEU_TEST.note = 'Lỗi: ' + e.message;
} finally {
  KIEU_TEST.running = false; KIEU_TEST.finishedAt = new Date().toISOString();
}
}

app.get('/api/marketing/kieu-test', (req, res) => {
  if (!SANDBOX_TOKEN) return res.json({ error: 'Chua khai SANDBOX_TOKEN' });
  const day = req.query.day || '2026-06-12';
  if (!KIEU_TEST.running && (req.query.restart === '1' || KIEU_TEST.results.lengt
    runKieuTest(day); // chạy ngầm, không await
  }
  res.json({
    huongDan: 'Mỗi kiểu cách nhau ~63s. Mở lại link này mỗi ~1 phút để xem kết qu
      + 'Kiểu nào có createOnDay gần bằng count chính là kiểu lọc theo NGÀY TẠO (
    running: KIEU_TEST.running, day: KIEU_TEST.day,
    startedAt: KIEU_TEST.startedAt, finishedAt: KIEU_TEST.finishedAt,
    daThu: KIEU_TEST.results.length + '/' + KIEU_LIST.length,
    results: KIEU_TEST.results, note: KIEU_TEST.note,
  });
});
});

```

```

/* ===== TRANG SẢN PHẨM =====
- Người quản lý sản phẩm: đồng bộ từ Google Sheet (publish ra CSV).
  Khai SHEET_CSV_URL = link CSV của sheet.
- Số contact + số đơn chốt theo sản phẩm: từ báo cáo Sandbox "MKT theo SP".
  (Dùng chung phiên đăng nhập web đã có. Nếu URL báo cáo khác, đặt SANDBOX_P
===== */
const SHEET_CSV_URL = process.env.SHEET_CSV_URL || '';
const SANDBOX_PRODUCT_REPORT_URL = process.env.SANDBOX_PRODUCT_REPORT_URL
  || 'https://api.sandbox.com.vn/report/api/Report/BaoCaoMktTheoSanPham';

const normProd = s => String(s == null ? '' : s).trim().toLowerCase().replace(/\\s

// CSV/TSV parser nhỏ - tự nhận dấu phân tách (phẩy, tab, hoặc chấm phẩy)
function parseCSV(text) {
  const firstLine = (text.split(/\\r?\\n/)[0] || '');
  const n = ch => (firstLine.split(ch).length - 1);
  let delim = ',';
  if (n('\\t') > n(delim)) delim = '\\t';
  if (n(';') > n(delim)) delim = ';';
  const rows = []; let row = [], cur = '', q = false;
  for (let i = 0; i < text.length; i++) {
    const c = text[i];
    if (q) { if (c === '"') { if (text[i + 1] === '"') { cur += '"'; i++; } else
    else if (c === '"') q = true;
    else if (c === delim) { row.push(cur); cur = ''; }
    else if (c === '\\n') { row.push(cur); rows.push(row); row = []; cur = ''; }
    else if (c === '\\r') { /* bỏ */ }
    else cur += c;
  }
  if (cur !== '' || row.length) { row.push(cur); rows.push(row); }
  return rows;
}

let OWNERS = {}, OWNERS_AT = 0; const OWNERS_TTL = 5 * 60 * 1000;
async function loadOwners(force) {
  if (!SHEET_CSV_URL) return OWNERS;
  if (!force && Object.keys(OWNERS).length && Date.now() - OWNERS_AT < OWNERS_TTL
  try {
    const r = await fetchWithTimeout(SHEET_CSV_URL, { redirect: 'follow' });
    const text = await r.text();
    const rows = parseCSV(text).filter(rw => rw.some(c => String(c).trim() !== ''
    if (!rows.length) return OWNERS;
    let pCol = 0, mCol = 1, start = 0;
    const hdr = rows[0].map(h => String(h).trim().toLowerCase());
    const find = keys => hdr.findIndex(h => keys.some(k => h.includes(k)));
    const pc = find(['sản phẩm', 'san pham', 'sanpham', 'product', 'tên sp', 'ten

```

```

const mc = find(['quản lý', 'quan ly', 'phụ trách', 'phu trach', 'nhân viên',
let gc = find(['giá nhập', 'gia nhap', 'gianhap', 'giá vốn', 'gia von']);
let dc = find(['ngày đăng', 'ngay dang', 'ngaydang', 'ngày tạo', 'ngay tao',
let tc = find(['giá thái', 'gia thai', 'giathai', 'thai price', 'giá thb', 'g
let cc = find(['mã sp thái', 'ma sp thai', 'mã thái', 'ma thai', 'thai code',
if (pc >= 0 && mc >= 0) { pCol = pc; mCol = mc; start = 1; }
if (gc < 0 && start === 1) gc = 3; // mặc định cột D = giá nhập
if (dc < 0 && start === 1) dc = 4; // mặc định cột E = ngày đăng
const toNum = v => { const n = parseInt(String(v == null ? '' : v).replace(/[
// Parse ngày DD/MM/YYYY hoặc DD-MM-YYYY -> 'YYYY-MM-DD' (null nếu trống/sai)
const parseDate = v => {
  const s = String(v == null ? '' : v).trim();
  if (!s) return null;
  const mm = s.match(/^(\\d{1,2})[\\/-\\.](\\d{1,2})[\\/-\\.](\\d{2,4})$/);
  if (!mm) return null;
  let d = mm[1], mo = mm[2], y = mm[3];
  y = y.length === 2 ? '20' + y : y;
  const dt = new Date(+y, +mo - 1, +d);
  return isNaN(dt.getTime()) ? null : dt.toISOString().slice(0, 10);
};
const map = {};
for (let i = start; i < rows.length; i++) {
  const praw = String(rows[i][pCol] || '').trim();
  const m = String(rows[i][mCol] || '').trim();
  const giaNhap = gc >= 0 ? toNum(rows[i][gc]) : 0;
  const giaThai = tc >= 0 ? toNum(rows[i][tc]) : 0;
  const maSPThai = cc >= 0 ? String(rows[i][cc] || '').trim().toUpperCase() :
  const ngayDang = dc >= 0 ? parseDate(rows[i][dc]) : null;
  const p = normProd(praw);
  if (p && (m || giaNhap)) map[p] = { manager: m, productRaw: praw, giaNhap,
}
  if (Object.keys(map).length) { OWNERS = map; OWNERS_AT = Date.now(); }
} catch (e) { /* giữ bản cũ nếu lỗi */ }
return OWNERS;
}

// Gọi báo cáo sản phẩm (dùng phiên đăng nhập web của máy chủ)
async function sandboxProductReport(since, until, extra = {}) {
  const tuNgay = `${since}T00:00:00+07:00`, denNgay = `${until}T23:59:59+07:00`;
  const payload = {
    strIdNguonDuLieu: null, kieuNgay: 'NgayTao', tuNgay, denNgay,
    idNhomSanPham: null, idSanPhamCha: null, idSanPham: null, unitCode: null,
    tiTrongChiaTinhTheo: 0, isChietKhau: true, isVat: true,
    idChiNhanh: SANDBOX_CHINHANH, typeViewDetail: null,
    idPhongBanSale: null, idNhomNhanVienSale: null, idUserSale: null,
    idPhongBanMkts: null, idNhomNhanVienMkts: null, idUserMkts: null,
    date: [tuNgay, denNgay], khoId: null,

```

```

    pageInfo: { page: 1, pageSize: 1000 }, sorts: [],
    ...extra
  });
const call = url => fetchWithTimeout(url, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'Accept': 'application/json',
  body: JSON.stringify(payload),
});
if (!sandboxCookie) await sandboxLogin();
let r = await call(SANDBOX_PRODUCT_REPORT_URL);
if (r.status === 401 || r.status === 403) { await sandboxLogin(); r = await call
const j = await r.json().catch(() => ({ success: false, message: 'Phản hồi không
return { httpStatus: r.status, json: j };
}

// Chuyển dữ liệu báo cáo sản phẩm -> bảng, ghép người quản lý từ Google Sheet
function mapProductReport(j, owners) {
  const d = (j && j.data) || {};
  // báo cáo trả về nhiều lưới; lấy lưới theo sản phẩm
  let arr = Array.isArray(d.productGridTable) ? d.productGridTable
    : Array.isArray(d.mainGridTable) ? d.mainGridTable : null;
  if (!Array.isArray(arr)) { for (const k of Object.keys(d)) { if (Array.isArray(
arr = arr || [];
const pick = (o, keys) => { for (const k of keys) if (o[k] !== null) return o[k]
return arr.map(o => {
  const name = pick(o, ['tenSanPham', 'tenSp', 'sanPham', 'itemName', 'productN
  const contact = +pick(o, ['soLuongContact', 'soLuongContactThucTe', 'soContac
  const chot = +pick(o, ['soLuongChotDon', 'soLuongChotDonThucTe', 'soDonChot',
  const own = owners[normProd(name)];
  return { product: String(name).trim(), manager: own ? own.manager : '', conta
});
}

// Soi cấu trúc báo cáo sản phẩm (từ máy chủ). Mở:
// /api/products/probe?since=2026-06-01&until=2026-06-17
// Trả về tên các lưới + tên cột + 1 dòng mẫu để chốt cột contact/đơn chốt.
app.get('/api/products/probe', async (req, res) => {
  const today = new Date().toISOString().slice(0, 10);
  const since = req.query.since || today, until = req.query.until || since;
  try {
    const rp = await sandboxProductReport(since, until);
    const d = (rp.json && rp.json.data) || {};
    const grids = {};
    for (const k of Object.keys(d)) {
      const v = d[k];
      if (Array.isArray(v)) grids[k] = { n: v.length, keys: v.length ? Object.key
      else if (v && typeof v === 'object') grids[k] = { obj: true, keys: Object.k

```

```

    }
    res.json({ httpStatus: rp.httpStatus, success: rp.json && (rp.json.success ??
    } catch (e) { res.json({ error: e.message }); }
  });

// Kiểm tra Google Sheet đọc được không. Mở /api/products/owners
app.get('/api/products/owners', async (req, res) => {
  if (!SHEET_CSV_URL) return res.json({ ok: false, message: 'Chưa khai SHEET_CSV_'
  try {
    const r = await fetch(SHEET_CSV_URL, { redirect: 'follow' });
    const text = await r.text();
    const rows = parseCSV(text).filter(rw => rw.some(c => String(c).trim() !== ''
    const o = await loadOwners(true);
    const sample = Object.values(o).slice(0, 8).map(x => ({ sanPham: x.productRaw
    res.json({
      ok: Object.keys(o).length > 0,
      soSanPham: Object.keys(o).length,
      tieuDe: rows[0] || [],
      soCot: (rows[0] || []).length,
      sample,
    });
  } catch (e) { res.json({ ok: false, message: e.message }); }
});

// Báo cáo sản phẩm cho trang Sản phẩm
app.get('/api/products/report', async (req, res) => {
  const today = new Date().toISOString().slice(0, 10);
  const since = req.query.since || today;
  const until = req.query.until || since;
  try {
    const owners = await loadOwners(false);
    const rp = await sandboxProductReport(since, until);
    if (!(rp.json && (rp.json.success ?? rp.json.Success)))
      return res.json({ ok: false, since, until, httpStatus: rp.httpStatus, messa
    let rows = mapProductReport(rp.json, owners);
    let managers = [...new Set(Object.values(owners).map(o => o.manager))].sort()
    // Quyền "product": chỉ thấy sản phẩm do chính mình quản lý
    const me = req.session.user || {};
    if (me.role === 'product') {
      const mine = normProd(me.manager || '');
      rows = rows.filter(r => normProd(r.manager) === mine && mine);
      managers = me.manager ? [me.manager] : [];
    }
    res.json({ ok: true, ver: 'prod-2026-06-15-v2', since, until, rows, managers,
  } catch (e) { res.json({ ok: false, since, until, message: e.message }); }
});

```

```

/* ===== TÍNH LƯƠNG PHÁT TRIỂN SẢN PHẨM =====
    Hoa hồng theo người Quản Lý SP (cột C Google Sheet):
    - SP mới (đăng <= 2 tháng tính đến cuối tháng lương): 1.0% × (Doanh thu - Giá
    - SP cũ (đăng > 2 tháng): 0.7% × (Doanh thu - Giá v
    - SP không có Ngày Đăng: KHÔNG tính lương
    Doanh thu = doanhSo từ Sandbox (theo SP)
    Giá vốn = soLuongSanPham (ship thành công) × giá nhập (cột D Sheet)
    ===== */
const HH_SP_MOI = Number(process.env.HH_SP_MOI || 0.01); // 1%
const HH_SP_CU = Number(process.env.HH_SP_CU || 0.007); // 0.7%

// BHHX mặc định cho nhân viên phát triển SP (theo tên Quản Lý trong Sheet)
const PTSP_BHHX = {
  'Đào Trung Kiên': 630000,
  'Nguyễn Huyền Trang': 577500,
};

// Dữ liệu nhập tay cho lương PTSP (thưởng/phạt) - lưu theo tháng, trong DATA_DIR
const PTSP_MANUAL_FILE = path.join(DATA_DIR, 'salary-product-manual.json');

// — Tài khoản ngân hàng nhân viên —
const BANK_ACCOUNTS_FILE = path.join(DATA_DIR, 'bank-accounts.json'); let BANK_ACC
try { BANK_ACCOUNTS = JSON.parse(fs.readFileSync(BANK_ACCOUNTS_FILE, 'utf8')) ||
function saveBankAccounts() {
  try { fs.writeFileSync(BANK_ACCOUNTS_FILE, JSON.stringify(BANK_ACCOUNTS)); } ca
}
app.get('/api/bank-accounts', (req, res) => {
  if (!req.session.user || req.session.user.role !== 'admin') return res.status(4
  res.json(BANK_ACCOUNTS);
});
app.post('/api/bank-accounts', (req, res) => {
  if (!req.session.user || req.session.user.role !== 'admin') return res.status(4
  const { name, bankBin, bankName, accountNo } = req.body || {};
  if (!name) return res.json({ ok: false, message: 'Thiếu tên' });
  const k = normProd(name);
  if (!bankBin && !accountNo) { delete BANK_ACCOUNTS[k]; }
  else { BANK_ACCOUNTS[k] = { name, bankBin: bankBin || '', bankName: bankName ||
  saveBankAccounts();
  res.json({ ok: true });
});

/* — SẢN PHẨM THỦ CÔNG (PTSP) —
    Những SP không có trên Sandbox (VD: chỉ bán Pushsale) nhưng vẫn cần tính lương
    Lưu theo tháng: PTSP_CUSTOM[month] = [{ product, manager, ngayDang }]
    DT/GV/Ship của SP này lấy hoàn toàn từ prodChannels (nhập tay).
const PTSP_CUSTOM_FILE = path.join(DATA_DIR, 'salary-product-custom.json');
let PTSP_CUSTOM = {};

```



```

try { PTSP_CUSTOM = JSON.parse(fs.readFileSync(PTSP_CUSTOM_FILE, 'utf8')) || {};
function savePtspCustom() {
  try { fs.writeFileSync(PTSP_CUSTOM_FILE, JSON.stringify(PTSP_CUSTOM)); }
  catch (e) { console.error('[SAVE] salary-product-custom lỗi:', e.message); }
}

app.get('/api/salary-product/custom', (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chỉ
  const month = req.query.month || '';
  res.json({ ok: true, month, list: PTSP_CUSTOM[month] || [] });
});

app.post('/api/salary-product/custom', express.json(), (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chỉ
  const { month, action, product, manager, ngayDang } = req.body || {};
  if (!month) return res.json({ ok: false, message: 'Thiếu tháng' });
  if (!PTSP_CUSTOM[month]) PTSP_CUSTOM[month] = [];
  const list = PTSP_CUSTOM[month];
  const pName = String(product || '').trim();
  const mName = String(manager || '').trim();
  if (!pName || !mName) return res.json({ ok: false, message: 'Thiếu tên SP hoặc
  const idx = list.findIndex(x => normProd(x.product) === normProd(pName) && norm

  if (action === 'delete') {
    if (idx >= 0) list.splice(idx, 1);
    // Xoá luôn DT/GV/ship nhập tay của SP này
    const mk = normProd(mName), pk = normProd(pName);
    if (PTSP_MANUAL[month] && PTSP_MANUAL[month][mk] && PTSP_MANUAL[month][mk].pr
      delete PTSP_MANUAL[month][mk].prodChannels[pk];
    savePtspCustom(); savePtspManual();
    return res.json({ ok: true, deleted: true });
  }

  const rec = { product: pName, manager: mName, ngayDang: ngayDang || null };
  if (idx >= 0) list[idx] = rec; else list.push(rec);
  savePtspCustom();
  res.json({ ok: true, list });
});

let PTSP_MANUAL = {};
try {
  PTSP_MANUAL = JSON.parse(fs.readFileSync(PTSP_MANUAL_FILE, 'utf8'));
  if (!Object.keys(PTSP_MANUAL).length) {
    try {
      const bak = JSON.parse(fs.readFileSync(PTSP_MANUAL_FILE + '.bak', 'utf8'));

```

```

        if (Object.keys(bak).length) { PTSP_MANUAL = bak; fs.writeFileSync(PTSP_MAN
    } catch {}
    }
} catch {
    try {
        const bak = JSON.parse(fs.readFileSync(PTSP_MANUAL_FILE + '.bak', 'utf8'));
        if (Object.keys(bak).length) { PTSP_MANUAL = bak; console.log('[RESTORE] sala
    } catch { PTSP_MANUAL = {}; }
}
function savePtspManual() {
    try {
        // — MERGE: đọc file đĩa, gộp RAM vào, rồi mới ghi —
        let disk = {};
        try { disk = JSON.parse(fs.readFileSync(PTSP_MANUAL_FILE, 'utf8')) || {}; } c

        for (const [month, emps] of Object.entries(PTSP_MANUAL)) {
            if (!disk[month]) disk[month] = {};
            for (const [key, val] of Object.entries(emps || {})) {
                disk[month][key] = val;
            }
        }

        const diskSize = JSON.stringify(disk).length;
        let bakSize = 0;
        try { bakSize = fs.statSync(PTSP_MANUAL_FILE + '.bak').size; } catch {}
        if (diskSize >= bakSize) {
            try { fs.writeFileSync(PTSP_MANUAL_FILE + '.bak', JSON.stringify(disk)); }
        }

        fs.writeFileSync(PTSP_MANUAL_FILE, JSON.stringify(disk));
        PTSP_MANUAL = disk;
    } catch (e) { console.error('[SAVE] salary-product-manual lỗi:', e.message); }
}

// API lấy/lưu dữ liệu tay cho lương PTSP
// Bỏ dấu tiếng Việt (để gộp key cũ "dao trung kien" với key mới "đào trung kiên"
function stripVN(s) {
    return String(s == null ? '' : s).normalize('NFD').replace(/[\u0300-\u036f]/g,
        .replace(/đ/g, 'd').replace(/Đ/g, 'D').toLowerCase().trim().replace(/\\s+/g, '
}
app.get('/api/salary-product/manual', (req, res) => {
    const month = req.query.month || '';
    const raw = PTSP_MANUAL[month] || {};
    const data = {};
    // GỘP dữ liệu theo tên bỏ dấu — gom cả key cũ (không dấu) và mới (có dấu)
    const byStrip = {};
    for (const [k, v] of Object.entries(raw)) {

```

```

const sk = stripVN(k);
if (!byStrip[sk]) byStrip[sk] = {};
// Gộp: ưu tiên giá trị khác 0 / có dữ liệu
const merged = byStrip[sk];
for (const [field, val] of Object.entries(v)) {
  if (field === 'channels') {
    merged.channels = merged.channels || {};
    for (const [ch, cv] of Object.entries(val || {})) {
      merged.channels[ch] = { ...(merged.channels[ch] || {}), ...cv };
    }
  } else if (field === 'prodChannels') {
    merged.prodChannels = merged.prodChannels || {};
    for (const [pk, pv] of Object.entries(val || {})) {
      merged.prodChannels[pk] = merged.prodChannels[pk] || {};
      for (const [ch, cv] of Object.entries(pv || {})) {
        merged.prodChannels[pk][ch] = { ...(merged.prodChannels[pk][ch] || {})
      }
    }
  } else if (val !== null && (merged[field] == null || merged[field] === 0)) {
    merged[field] = val;
  }
}
}

// Đưa dữ liệu đã gộp vào key CHUẨN (normProd = có dấu) theo tên trong PTSP_BHX
for (const name of Object.keys(PTSP_BHXH)) {
  const sk = stripVN(name);
  const k = normProd(name);
  if (byStrip[sk]) { data[k] = { ...byStrip[sk], name }; delete byStrip[sk]; }
}

// Các key còn lại (người không có trong PTSP_BHXH) giữ nguyên theo normProd
for (const [sk, v] of Object.entries(byStrip)) {
  data[v.name ? normProd(v.name) : sk] = v;
}

// Điền BHXH mặc định
for (const [name, bxxh] of Object.entries(PTSP_BHXH)) {
  const k = normProd(name);
  if (!data[k]) data[k] = { name, thuongSP: 0, thuongThang: 0, phat: 0, bxxh };
  else if (!data[k].bxxh) data[k].bxxh = bxxh;
}

res.json({ month, data });
});

app.post('/api/salary-product/manual', express.json(), (req, res) => {
  const { month, key, field, value, channel, metric } = req.body || {};
  if (!month || !key) return res.json({ ok: false, message: 'Thiếu tham số' });
  if (!PTSP_MANUAL[month]) PTSP_MANUAL[month] = {};
  // DỌN KEY TRÙNG: nếu tồn tại key cũ không dấu khác key hiện tại nhưng cùng tên
  // gộp dữ liệu cũ vào key chuẩn rồi xoá key cũ (tránh phân mảnh dữ liệu)

```

```

const stripKey = s => String(s||'').normalize('NFD').replace(/[\u0300-\u036f]/g
const targetStrip = stripKey(key);
for (const ek of Object.keys(PTSP_MANUAL[month])) {
  if (ek !== key && stripKey(ek) === targetStrip) {
    const oldRec = PTSP_MANUAL[month][ek];
    if (!PTSP_MANUAL[month][key]) PTSP_MANUAL[month][key] = {};
    // Gộp field cũ vào key mới (không ghi đè field mới đã có giá trị)
    for (const [f, vv] of Object.entries(oldRec)) {
      if (f === 'channels') {
        PTSP_MANUAL[month][key].channels = PTSP_MANUAL[month][key].channels ||
        for (const [ch, cv] of Object.entries(vv || {}))
          PTSP_MANUAL[month][key].channels[ch] = { ...(PTSP_MANUAL[month][key].
      } else if (f === 'prodChannels') {
        PTSP_MANUAL[month][key].prodChannels = PTSP_MANUAL[month][key].prodChan
        for (const [pk, pv] of Object.entries(vv || {})) {
          PTSP_MANUAL[month][key].prodChannels[pk] = PTSP_MANUAL[month][key].pr
          for (const [ch, cv] of Object.entries(pv || {}))
            PTSP_MANUAL[month][key].prodChannels[pk][ch] = { ...(PTSP_MANUAL[mo
          }
        } else if (PTSP_MANUAL[month][key][f] == null || PTSP_MANUAL[month][key][
          PTSP_MANUAL[month][key][f] = vv;
        }
      }
    }
    delete PTSP_MANUAL[month][ek];
  }
}
if (!PTSP_MANUAL[month][key]) PTSP_MANUAL[month][key] = {};
const num = parseInt(String(value == null ? '' : value).replace(/^[^d]/g, ''));
const { product: prodName } = req.body || {};
if (prodName && channel && metric) {
  // Lưu DT/GV/Ship theo sản phẩm + kênh: prodChannels["tên sp"]["thailan"] = {
  const ch = ptspChan(channel); // 'shopee' → 'tmdt'
  if (!PTSP_MANUAL[month][key].prodChannels) PTSP_MANUAL[month][key].prodChanne
  const pk = normProd(prodName);
  if (!PTSP_MANUAL[month][key].prodChannels[pk]) PTSP_MANUAL[month][key].prodCh
  if (!PTSP_MANUAL[month][key].prodChannels[pk][ch]) PTSP_MANUAL[month][key].pr
  PTSP_MANUAL[month][key].prodChannels[pk][ch][metric] = num;
  // Xoá entry nếu dt, gv, ship đều = 0
  const entry = PTSP_MANUAL[month][key].prodChannels[pk][ch];
  if (!(+entry.dt || 0) && !(+entry.gv || 0) && !(+entry.ship || 0)) {
    delete PTSP_MANUAL[month][key].prodChannels[pk][ch];
    if (!Object.keys(PTSP_MANUAL[month][key].prodChannels[pk]).length)
      delete PTSP_MANUAL[month][key].prodChannels[pk];
  }
} else if (channel && metric) {
  // Lưu kênh nhập tay tổng (giữ tương thích): channels.thailan.dt / channels.t
  if (!PTSP_MANUAL[month][key].channels) PTSP_MANUAL[month][key].channels = {};

```

```

    if (!PTSP_MANUAL[month][key].channels[channel]) PTSP_MANUAL[month][key].chann
    PTSP_MANUAL[month][key].channels[channel][metric] = num;
  } else if (field) {
    PTSP_MANUAL[month][key][field] = num;
  } else {
    return res.json({ ok: false, message: 'Thiếu field hoặc channel/metric' });
  }
  savePtspManual();
  res.json({ ok: true });
});

```

```

// Các kênh nhập tay cho PTSP và tỷ lệ hoa hồng cố định 0.7%
// Kênh doanh thu nhập tay. 'shopee' cũ được đổi tên thành 'tmdt' (giữ tương thích
const PTSP_CHANNELS = ['thailan', 'pushsale', 'tmdt', 'dt85'];
// Map kênh cũ → mới (dữ liệu đã lưu trước đây dùng key 'shopee')
const PTSP_CHANNEL_ALIAS = { shopee: 'tmdt' };
const ptspChan = c => PTSP_CHANNEL_ALIAS[c] || c;
const HH_KENH_KHAC = Number(process.env.HH_KENH_KHAC || 0.007); // 0.7%
const PHI_SHIP_DON_PTSP = Number(process.env.PHI_SHIP_DON || 30000); // phí ship

```

```

/* Dụng báo cáo lương PTSP – tách thành hàm để trang "Kết quả kinh doanh" dùng lại
async function buildPtspReport(since, until, me) {

```

```

  me = me || { role: 'admin' };

```

```

  {
    const owners = await loadOwners(false);

```

```

    const statusParam = process.env.SALARY_STATUS_PARAM || 'giaoHangTrangThaiMa';
    const giaoVal = process.env.SALARY_VAL_GIAO || '31';
    const ttVal = process.env.SALARY_VAL_TT || '32';
    const castVal = v => (v !== '' && !isNaN(+v)) ? +v : v;

```

```

    const [rpGiao, rpTT] = await Promise.all([
      sandboxProductReport(since, until, { [statusParam]: castVal(giaoVal) }),
      sandboxProductReport(since, until, { [statusParam]: castVal(ttVal) })
    ]);

```

```

    if (!(rpGiao.json && (rpGiao.json.success ?? rpGiao.json.Success)) ||
      !(rpTT.json && (rpTT.json.success ?? rpTT.json.Success))) {
      return { ok: false, since, until, message: 'Lỗi gọi báo cáo sản phẩm Sandbo
    }

```

```

    const pick = (o, keys) => { for (const k of keys) if (o[k] !== null) return o[

```

```

    const mergedMap = {};

```

```

    const processArr = (arr) => {

```

```

      for (const o of arr) {
        const name = String(pick(o, ['tenSanPham', 'tenSp', 'sanPham', 'name']) |

```

```

    if (!name) continue;
    const doanhThu = +pick(o, ['doanhSo', 'doanhThu', 'tongDoanhSo']) || 0;
    const soLuongSP = +pick(o, ['soLuongSanPham', 'soLuongSP', 'soLuong']) || 0;
    const soDon = +pick(o, ['soLuongChotDon', 'soLuongChotDonThucTe', 'soDonC

const key = normProd(name);
if (!mergedMap[key]) {
    mergedMap[key] = { name, doanhThu: 0, soLuongSP: 0, soDon: 0 };
}
mergedMap[key].doanhThu += doanhThu;
mergedMap[key].soLuongSP += soLuongSP;
mergedMap[key].soDon += soDon;
}
};

const dGiao = (rpGiao.json && rpGiao.json.data) || {};
processArr(Array.isArray(dGiao.productGridTable) ? dGiao.productGridTable : [

const dTT = (rpTT.json && rpTT.json.data) || {};
processArr(Array.isArray(dTT.productGridTable) ? dTT.productGridTable : []);

const mergedArr = Object.values(mergedMap);

// Mốc phân loại mới/cũ: cuối tháng của "until"
const endOfMonth = new Date(until.slice(0, 7) + '-01');
endOfMonth.setMonth(endOfMonth.getMonth() + 1);
endOfMonth.setDate(0); // ngày cuối tháng
// 2 tháng trước cuối tháng → mốc; đăng SAU mốc = mới, TRƯỚC = cũ
const twoMonthsAgo = new Date(endOfMonth);
twoMonthsAgo.setMonth(twoMonthsAgo.getMonth() - 2);

const products = mergedArr.map(o => {
    const name = o.name;
    const doanhThu = o.doanhThu;
    const soLuongSP = o.soLuongSP;
    const soDon = o.soDon;
    const own = owners[normProd(name)];
    const manager = own ? own.manager : '';
    const giaNhap = own ? (own.giaNhap || 0) : 0;
    const ngayDang = own ? own.ngayDang : null;
    const giaVon = soLuongSP * giaNhap;
    const phiShip = soDon * PHI_SHIP_DON_PTSP; // phí ship = số đơn × 30.000
    const loiNhuon = doanhThu - giaVon - phiShip;

    // Phân loại mới/cũ theo ngày đăng
    let loai = 'khong-tinh'; // mặc định: không có ngày đăng → không tính
    let tyLe = 0;

```

```

    if (ngayDang) {
      const dt = new Date(ngayDang);
      // đăng <= 2 tháng (sau hoặc bằng mốc twoMonthsAgo) = mới
      loai = (dt >= twoMonthsAgo) ? 'moi' : 'cu';
      tyLe = (loai === 'moi') ? HH_SP_MOI : HH_SP_CU;
    }

    const hoaHong = Math.round(loiNhuon * tyLe);

    return { product: name, manager, doanhThu, autoDoanhThu: doanhThu, soLuongS
  }).filter(p => p.product);

// — Thêm SẢN PHẨM THỦ CÔNG (không có trên Sandbox) —
const monthKeyPC = (since || '').slice(0, 7);
const customList = PTSP_CUSTOM[monthKeyPC] || [];
for (const cp of customList) {
  const pkC = normProd(cp.product);
  // Bỏ qua nếu Sandbox đã có SP này rồi (tránh trùng)
  if (products.some(p => normProd(p.product) === pkC)) continue;
  const own = owners[pkC];
  // Ưu tiên thông tin từ Google Sheet nếu có, không thì dùng dữ liệu nhập ta
  const manager = (own && own.manager) || cp.manager || '';
  const ngayDang = cp.ngayDang || (own ? own.ngayDang : null);
  // SP thêm thủ công MẶC ĐỊNH là SP MỚI (1%).
  // Chỉ tính là SP cũ khi có ngày đăng và ngày đó đã quá 2 tháng.
  let loai = 'moi', tyLe = HH_SP_MOI;
  if (ngayDang) {
    const dt = new Date(ngayDang);
    loai = (dt >= twoMonthsAgo) ? 'moi' : 'cu';
    tyLe = (loai === 'moi') ? HH_SP_MOI : HH_SP_CU;
  }
  products.push({
    product: cp.product, manager,
    doanhThu: 0, autoDoanhThu: 0, soLuongSP: 0, soDon: 0,
    giaNhap: own ? (own.giaNhap || 0) : 0,
    giaVon: 0, autoGiaVon: 0, phiShip: 0, autoPhiShip: 0,
    loiNhuon: 0, ngayDang, loai, tyLe, hoaHong: 0,
    chDT: {}, chGV: {}, chSHIP: {}, isCustom: true,
  });
}

// Gộp DT/GV/Ship nhập tay theo sản phẩm+kênh vào từng product
const manualMonthPC = PTSP_MANUAL[monthKeyPC] || {};
for (const p of products) {
  const mgrKey = normProd(p.manager);
  const rec = manualMonthPC[mgrKey] || {};
  const pc = rec.prodChannels || {};
  const pk = normProd(p.product);

```

```

const prodCh = pc[pk];
if (prodCh) {
  let addDT = 0, addGV = 0, addSHIP = 0;
  for (const [cidRaw, cv] of Object.entries(prodCh)) {
    const cid = ptspChan(cidRaw); // 'shopee' (cũ) → 'tmdt'
    const cdt = +((cv || {}).dt) || 0;
    const cgv = +((cv || {}).gv) || 0;
    const cship = +((cv || {}).ship) || 0;
    addDT += cdt; addGV += cgv; addSHIP += cship;
    p.chDT[cid] = (p.chDT[cid] || 0) + cdt;
    p.chGV[cid] = (p.chGV[cid] || 0) + cgv;
    p.chSHIP[cid] = (p.chSHIP[cid] || 0) + cship;
  }
  p.doanhThu += addDT;
  p.giaVon += addGV;
  p.phiShip += addSHIP;
  p.loiNhuan = p.doanhThu - p.giaVon - p.phiShip;
  p.hoaHong = Math.round(p.loiNhuan * p.tyLe);
}
}

// Quyền: tài khoản "product" chỉ thấy SP của mình
// 'me' là tham số của hàm – dùng để lọc quyền xem
let visible = products;
if (me.role === 'product') {
  const mine = normProd(me.manager || '');
  visible = products.filter(p => normProd(p.manager) === mine && mine);
}

// Tổng hợp hoa hồng theo người Quản Lý
const byManager = {};
for (const p of visible) {
  const mgr = p.manager || '(chưa gán)';
  if (!byManager[mgr]) byManager[mgr] = { manager: mgr, soSP: 0, soSPmoi: 0,
  const m = byManager[mgr];
  m.soSP++;
  if (p.loai === 'moi') m.soSPmoi++;
  else if (p.loai === 'cu') m.soSPcu++;
  m.soDon += p.soDon;
  m.soLuongSP += p.soLuongSP;
  m.doanhThu += p.doanhThu;
  m.giaVon += p.giaVon;
  m.phiShip += p.phiShip;
  m.loiNhuan += p.loiNhuan;
  m.hoaHong += p.hoaHong;
}
}

```



```

// Gắn BHXH mặc định + tính HH kênh khác (Thái Lan, Pushsale, Shopee × 0.7%)
const monthKey = (since || '').slice(0, 7);
const manualMonth = PTSP_MANUAL[monthKey] || {};
for (const m of Object.values(byManager)) {
  m.bhxh = PTSP_BHXH[m.manager] || 0;
  // Lấy channels nhập tay của người này
  const rec = manualMonth[normProd(m.manager)] || {};
  const ch = rec.channels || {};
  let dtKhac = 0, gvKhac = 0, shipKhac = 0;
  m.channels = {};
  for (const cid of PTSP_CHANNELS) {
    const dt = Number((ch[cid] || {}).dt) || 0;
    const gv = Number((ch[cid] || {}).gv) || 0;
    const ship = Number((ch[cid] || {}).ship) || 0;
    m.channels[cid] = { dt, gv, ship };
    dtKhac += dt; gvKhac += gv; shipKhac += ship;
  }
  m.dtKhac = dtKhac;
  m.gvKhac = gvKhac;
  m.shipKhac = shipKhac;
  // Hoa hồng kênh khác = (DT - GV - Phí ship) × 0.7%
  m.hoaHongKhac = Math.round((dtKhac - gvKhac - shipKhac) * HH_KENH_KHAC);
  // Tổng hoa hồng = HH Sandbox (đã có) + HH kênh khác
  m.hoaHongSandbox = m.hoaHong;
  m.hoaHong = m.hoaHong + m.hoaHongKhac;
  // Cộng doanh thu/giá vốn/phi ship kênh khác vào tổng hiển thị
  m.doanhThuTong = m.doanhThu + dtKhac;
  m.giaVonTong = m.giaVon + gvKhac;
  m.phiShipTong = (m.phiShip || 0) + shipKhac;
}
const managers = Object.values(byManager).sort((a, b) => b.hoaHong - a.hoaHong);
const total = managers.reduce((s, m) => ({
  soSP: s.soSP + m.soSP, soDon: s.soDon + m.soDon, soLuongSP: s.soLuongSP + m
  doanhThu: s.doanhThu + m.doanhThu, giaVon: s.giaVon + m.giaVon, phiShip: s.
  loiNhuan: s.loiNhuan + m.loiNhuan, hoaHong: s.hoaHong + m.hoaHong,
}), { soSP: 0, soDon: 0, soLuongSP: 0, doanhThu: 0, giaVon: 0, phiShip: 0, loiNhuan: 0, hoaHong: 0 });

// Cập nhật total gồm cả kênh khác
total.dtKhac = managers.reduce((s, m) => s + (m.dtKhac || 0), 0);
total.gvKhac = managers.reduce((s, m) => s + (m.gvKhac || 0), 0);
total.shipKhac = managers.reduce((s, m) => s + (m.shipKhac || 0), 0);
total.hoaHongKhac = managers.reduce((s, m) => s + (m.hoaHongKhac || 0), 0);
total.doanhThu += total.dtKhac;
total.giaVon += total.gvKhac;
total.phiShip = (total.phiShip || 0) + total.shipKhac;

return {

```

```

        ok: true, since, until,
        tyLeMoi: HH_SP_MOI, tyLeCu: HH_SP_CU, tyLeKhac: HH_KENH_KHAC,
        channels: PTSP_CHANNELS,
        mocPhanLoai: twoMonthsAgo.toISOString().slice(0, 10),
        managers, products: visible, total,
        ownersCount: Object.keys(owners).length,
        me: { role: me.role, manager: me.manager || '' },
        lastUpdated: new Date().toISOString(),
    };
}
}

app.get('/api/salary-product/report', async (req, res) => {
    const today = new Date().toISOString().slice(0, 10);
    const since = req.query.since || today;
    const until = req.query.until || since;
    try {
        res.json(await buildPtspReport(since, until, req.session.user || {}));
    } catch (e) {
        const msg = e.name === 'AbortError' ? 'API Sandbox không phản hồi (timeout 30'
        res.json({ ok: false, since, until, message: msg });
    }
});

/* ===== TÍNH LƯƠNG MARKETING =====
Lương = (Doanh thu - Chi phí QC - Giá vốn - Phí ship) × 2%
- Doanh thu = doanh số (Đã giao hàng) + doanh số (Đã thanh toán)
- Chi phí QC = chỉ tiêu Meta theo từng nhân viên
- Giá vốn = Σ(số lượng SP × giá nhập - cột D Google Sheet)
- Phí ship = (số đơn đã giao + đã thanh toán) × 30.000
Tham số trạng thái của báo cáo tìm bằng /api/salary/probe, rồi khai:
    SALARY_STATUS_PARAM (mặc định loaiDoanhSo), SALARY_VAL_GIAO, SALARY_VAL_TT
===== */

const LUONG_TY_LE = Number(process.env.LUONG_TY_LE || 0.02);
const PHI_SHIP_DON = Number(process.env.PHI_SHIP_DON || 30000);
const QC_TAX = Number(process.env.QC_TAX || 0.11); // 11% thuế VAT cộng vào chi
// Chỉ hiện nhân viên trong danh sách EMPLOYEES + "Admin"; tên khác (vd khách lẻ)
const SALARY_ALLOW = new Set([...EMPLOYEES.map(e => normProd(e.full)), normProd('
const SALARY_STATUS_PARAM = process.env.SALARY_STATUS_PARAM || 'giaoHangTrangThai
const SALARY_VAL_GIAO = process.env.SALARY_VAL_GIAO || '31'; // 31 = Đã giao hàn
const SALARY_VAL_TT = process.env.SALARY_VAL_TT || '32'; // 32 = Đã thanh to
const castVal = v => (v !== '' && !isNaN(+v)) ? +v : v;

// Báo cáo lead theo nhân sự + tham số lọc thêm (vd trạng thái giao/thanh toán)
async function sandboxReportEx(since, until, extra) {
    const tuNgay = `${since}T00:00:00.000+07:00`, denNgay = `${until}T23:59:59.998+
    const payload = {

```

```

    pageInfo: { page: 1, pageSize: 1000 }, sorts: [],
    kieuXem: 4, loaiNhanVien: 1, isChietKhau: true, isVat: true,
    date: [tuNgay, denNgay], tuNgay, denNgay,
    idChiNhanh: SANDBOX_CHINHANH, kieuNgay: 'NgayTao',
    typeViewDetail: null, idPhongBanSale: null, idNhomNhanVienSale: null, idUsers
    idPhongBanMkts: null, idNhomNhanVienMkts: null, idUserMkts: null, ...(extra |
  );
  const call = () => fetchWithTimeout(SANDBOX_REPORT_URL, { method: 'POST', head
  if (!sandboxCookie) await sandboxLogin();
  let r = await call();
  if (r.status === 401 || r.status === 403) { await sandboxLogin(); r = await cal
  return r.json().catch(() => ({ success: false }));
}
function reportRowsEx(j) {
  const d = (j && j.data) || {};
  return (d.reportLeadByNhanSuMktDtos || []).map(r => ({
    name: (r.ten || '').trim(), id: r.marketingUserId,
    doanhthu: +r.doanhSo || 0,
    soDon: +(r.soDonHang ?? r.soDonChot ?? 0) || 0,
    soSP: +r.soLuongSanPham || 0,
  }));
}

// Số lượng từng SP theo 1 nhân viên marketing (có thể kèm lọc trạng thái) – để t
async function productQtyByUser(since, until, marketingUserId, extra) {
  const tuNgay = `${since}T00:00:00+07:00`, denNgay = `${until}T23:59:59+07:00`;
  const payload = { strIdNguonDuLieu: null, kieuNgay: 'NgayTao', tuNgay, denNgay,
  const call = () => fetchWithTimeout(SANDBOX_PRODUCT_REPORT_URL, { method: 'POST
  let r = await call();
  if (r.status === 401 || r.status === 403) { await sandboxLogin(); r = await cal
  const j = await r.json().catch(() => ({}));
  const d = (j && j.data) || {};
  const arr = Array.isArray(d.productGridTable) ? d.productGridTable : [];
  return arr.map(o => ({ ten: (o.tenSanPham || '').trim(), soLuong: +o.soLuongSan
}

// Gọi báo cáo sản phẩm + tham số lọc thêm; trả tổng doanh số & số lượng
async function productReportEx(since, until, extra) {
  const tuNgay = `${since}T00:00:00+07:00`, denNgay = `${until}T23:59:59+07:00`;
  const payload = { strIdNguonDuLieu: null, kieuNgay: 'NgayTao', tuNgay, denNgay,
  const call = () => fetchWithTimeout(SANDBOX_PRODUCT_REPORT_URL, { method: 'POST
  if (!sandboxCookie) await sandboxLogin();
  let r = await call();
  if (r.status === 401 || r.status === 403) { await sandboxLogin(); r = await cal
  const j = await r.json().catch(() => ({}));
  const d = (j && j.data) || {};
  const arr = Array.isArray(d.productGridTable) ? d.productGridTable : [];

```

```
    return { doanhSo: arr.reduce((s, o) => s + (+o.doanhSo || 0), 0), qty: arr.reduce((s, o) => s + (+o.qty || 0), 0) }
}
```

```
// Chi tiêu Meta theo từng nhân viên (dùng chung getCampaigns: có thù lại + đệm a
async function metaSpendByEmployee(since, until) {
    const campaigns = await getCampaigns(since, until);
    const spend = {};
    for (const c of campaigns) {
        const k = normProd(c.employee || '');
        spend[k] = (spend[k] || 0) + (c.daily || []).reduce((a, d) => a + (d && d.spe
    }
    return spend;
}
```

```
// Tìm tham số trạng thái: /api/salary/probe?param=loaiDoanhSo&vals=1,2,3,4
app.get('/api/salary/probe', async (req, res) => {
    const since = req.query.since || '2026-05-01', until = req.query.until || '2026-05-01';
    const param = req.query.param || 'loaiDoanhSo';
    const vals = String(req.query.vals || '1,2,3,4').split(',').map(s => s.trim()).
    try {
        if (!sandboxCookie) await sandboxLogin();
        const out = [];
        for (const v of vals) {
            const rows = reportRowsEx(await sandboxReportEx(since, until, { [param]: ca
            out.push({ value: v, tongDoanhThu: rows.reduce((s, r) => s + r.doanhthu, 0)
        }
        res.json({ since, until, param, ketqua: out, goiY: 'Tìm value cho tổng = 6538
    } catch (e) { res.json({ error: e.message }); }
});
```

```
// Dò trạng thái trên BÁO CÁO SẢN PHẨM:
// /api/products/probe-status?names=trangThaiGiaoHang,trangThai&vals=31,32,1,2,
// arr=1 -> gửi giá trị dạng mảng [v]; báo cáo SP có lọc trạng thái nên 1 trong s
// sẽ cho tongDoanhSo ≈ 653.897.000 (đã giao) hoặc 730.168.200 (đã thanh toán).
app.get('/api/products/probe-status', async (req, res) => {
    const since = req.query.since || '2026-05-01', until = req.query.until || '2026-05-01';
    const names = String(req.query.names || 'trangThaiGiaoHang,trangThaiDonHang,tra
    const vals = String(req.query.vals || '31').split(',').map(s => s.trim()).filte
    const arr = req.query.arr === '1';
    try {
        if (!sandboxCookie) await sandboxLogin();
        const base = await productReportEx(since, until, {});
        const out = [];
        for (const nm of names) {
            for (const v of vals) {
                const val = castVal(v);
                const t = await productReportEx(since, until, { [nm]: arr ? [val] : val }
            }
        }
    } catch (e) { res.json({ error: e.message }); }
});
```

```

        out.push({ param: nm, value: v, tongDoanhSo: t.doanhSo, khacBase: t.doanh
    }
}
res.json({ since, until, dangMang: arr, base: base.doanhSo, ketqua: out, goiY
} catch (e) { res.json({ error: e.message }); }
});

```

```

// Dữ liệu MẪU tháng 5/2026 – lấy từ báo cáo "lead theo nhân sự" lọc Đã giao + Đã
// thanh toán (đã gộp 2 trạng thái). Dừng khi chưa cấu hình SALARY_VAL_GIAO/TT, đ
// xem ngay bảng lương tháng 5. doanhthu = doanh số gộp; soDon, soSP = gộp 2 trạn
const SALARY_SAMPLE = {

```

```

    '2026-05': {
        'Trịnh Đức Phương': { doanhthu: 554333000, soDon: 937, soSP: 1734 },
        'Nguyễn Thị Trà My': { doanhthu: 151416000, soDon: 243, soSP: 465 },
        'Tạ Quang Trường': { doanhthu: 147853000, soDon: 299, soSP: 846 },
        'Vũ Hà Giang': { doanhthu: 127568200, soDon: 211, soSP: 766 },
        'Lê Thị Ánh': { doanhthu: 105231000, soDon: 232, soSP: 320 },
        'Đoàn Việt Hà': { doanhthu: 98935000, soDon: 206, soSP: 269 },
        'Nguyễn Duy Huân': { doanhthu: 72695000, soDon: 137, soSP: 273 },
        'Nguyễn Trung Hiếu': { doanhthu: 65165000, soDon: 137, soSP: 381 },
        'Dương Văn Minh': { doanhthu: 55029000, soDon: 79, soSP: 139 },
    },
};

```

```

// Bảng lương: /api/salary/report?since=2026-05-01&until=2026-05-31
/* Dựng báo cáo lương MKT (dữ liệu TỰ ĐỘNG) – tách thành hàm để trang
    "Kết quả kinh doanh" dùng lại, không phải gọi HTTP vòng vo. */
async function buildSalaryReport(since, until) {
    const useConfig = !(SALARY_VAL_GIAO && SALARY_VAL_TT);
    const sample = SALARY_SAMPLE[since.slice(0, 7)];
    if (!useConfig && !sample)
        return { ok: false, since, until, message: 'Hiện chỉ có sẵn dữ liệu mẫu tháng
    {
        await sandboxLogin(); // luôn đăng nhập lại để đảm bảo cookie còn hạn
        const p = SALARY_STATUS_PARAM;
        const map = {};
        const add = arr => { for (const r of arr) { if (!r.name) continue; const k =

        let meta;
        if (useConfig) {
            // Lấy doanh thu thật theo 2 trạng thái (đã giao + đã thanh toán)
            const [giaoJ, ttJ, m] = await Promise.all([
                sandboxReportEx(since, until, { [p]: castVal(SALARY_VAL_GIAO) }),
                sandboxReportEx(since, until, { [p]: castVal(SALARY_VAL_TT) }),
                metaSpendByEmployee(since, until),
            ]);
            meta = m; add(reportRowsEx(giaoJ)); add(reportRowsEx(ttJ));

```

```

} else {
  // Dùng dữ liệu mẫu; lấy marketingUserId từ báo cáo mặc định (để tính giá v
  const [baseJ, m] = await Promise.all([
    sandboxReportEx(since, until, {}),
    metaSpendByEmployee(since, until),
  ]);
  meta = m;
  const idByName = {}; for (const r of reportRowsEx(baseJ)) idByName[normProd
    for (const name in sample) { const s = sample[name], k = normProd(name); ma
  }

const owners = await loadOwners(false);
const priceOf = ten => { const o = owners[normProd(ten)]; return o ? (o.giaNh
const emps = Object.values(map).filter(e => SALARY_ALLOW.has(normProd(e.name)
// Giá vốn CHÍNH XÁC: số lượng từng SP theo đúng trạng thái (giao + thanh toá
const giaoVal = castVal(SALARY_VAL_GIAO), ttVal = castVal(SALARY_VAL_TT);
const tasks = [];
for (const e of emps) {
  tasks.push(e.id ? productQtyByUser(since, until, e.id, { [p]: giaoVal }) :
  tasks.push(e.id ? productQtyByUser(since, until, e.id, { [p]: ttVal }) : Pr
}
const lists = await Promise.all(tasks);

// — Cấu hình Team Lead —————
// Hoa hồng leader = ½ × tổng lương 2% của nhân viên trong nhóm
// (KHÔNG tính lương của chính team lead)
const TEAM_LEAD = {
  'Trịnh Đức Phương': ['Đoàn Việt Hà', 'Nguyễn Duy Huân', 'Vũ Thuý An'],
  'Tạ Quang Trường': ['Nguyễn Thị Trà My', 'Dương Văn Minh', 'Lê Thị Ánh'],
};

// Tính lương 2% từng người trước, lưu vào lươngByName
const lươngByName = {};
const rows = emps.map((e, i) => {
  let giaVon = 0;
  for (const pr of lists[i * 2]) giaVon += pr.soLuong * priceOf(pr.ten);
  for (const pr of lists[i * 2 + 1]) giaVon += pr.soLuong * priceOf(pr.ten);
  giaVon = Math.round(giaVon);
  const chiPhiQC = Math.round((meta[normProd(e.name)] || 0) * (1 + QC_TAX));
  const phiShip = e.soDon * PHI_SHIP_DON;
  const lương = Math.round((e.doanhthu - chiPhiQC - giaVon - phiShip) * LUONG
  lươngByName[e.name] = lương;
  return { name: e.name, doanhthu: e.doanhthu, chiPhiQC, giaVon, phiShip, soD
});

// Gắn hoa hồng leader = ½ × tổng lương 2% của nhân viên trong nhóm
for (const row of rows) {

```

```

    const members = TEAM_LEAD[row.name];
    if (!members) continue;
    const tongLuong = members.reduce((s, name) => s + (luongByName[name] || 0),
    row.hoaHongLeader = Math.round(tongLuong / 2);
}

// — Thuởng DTT (tự động theo bảng bậc thang) —————
// DTT = Doanh thu - Giá vốn - Phí ship (KHÔNG trừ chi phí QC)
// Bậc thang: mỗi 90tr từ 150tr → +1tr lương cứng, sau 690tr mỗi 90tr thêm +1
const DTT_BRACKETS = [
  { min: 150e6, max: 240e6, bonus: 1e6 },
  { min: 240e6, max: 330e6, bonus: 2e6 },
  { min: 330e6, max: 420e6, bonus: 3e6 },
  { min: 420e6, max: 510e6, bonus: 4e6 },
  { min: 510e6, max: 600e6, bonus: 5e6 },
  { min: 600e6, max: 690e6, bonus: 6e6 },
];
const DTT_BASE = 690e6, DTT_STEP = 90e6, DTT_BONUS_STEP = 1e6, DTT_START_BONU

function tinhThuongDTT(dtt) {
  if (dtt < 150e6) return 0;
  for (const b of DTT_BRACKETS) {
    if (dtt >= b.min && dtt <= b.max) return b.bonus;
  }
  if (dtt > DTT_BASE) {
    const extra = Math.floor((dtt - DTT_BASE) / DTT_STEP);
    return DTT_START_BONUS + (extra + 1) * DTT_BONUS_STEP;
  }
  return 0;
}

for (const row of rows) {
  const dtt = row.doanhthu - row.giaVon - row.phiShip;
  row.dtt = Math.round(dtt);
  row.thuongDTT = tinhThuongDTT(dtt);
}

// — Thuởng Top 1 (tự động - người có lương 2% cao nhất) —————
// Chỉ 1 người, thưởng = ½ × lương 2% của chính họ (chỉ khi lương > 0)
const posRows = rows.filter(r => r.luong > 0);
if (posRows.length > 0) {
  posRows.sort((a, b) => b.luong - a.luong);
  const top1 = posRows[0];
  // Kiểm tra không có người nào bằng điểm (nếu bằng thì không ai được)
  const isUnique = posRows.length === 1 || posRows[0].luong !== posRows[1].lu
  if (isUnique) top1.thuongTop1 = Math.round(top1.luong / 2);
}

```

```

    rows.sort((a, b) => b.luong - a.luong);
    const roster = EMPLOYEES.filter(e => e.code).map(e => e.full).concat('Admin')
    return { ok: true, since, until, nguồn: useConfig ? 'live' : 'mau-t5', tyLe:
  }
}

```

```

app.get('/api/salary/report', async (req, res) => {
  const today = new Date().toISOString().slice(0, 10);
  const since = req.query.since || today, until = req.query.until || since;
  try {
    res.json(await buildSalaryReport(since, until));
  } catch (e) {
    const msg = e.name === 'AbortError' ? 'API Sandbox không phản hồi (timeout 30
    res.json({ ok: false, since, until, message: msg });
  }
});

```

// Chẩn đoán chi phí QC: /api/salary/debug?since=2026-05-01&until=2026-05-31
 // Cho biết từng tài khoản Meta còn chạy không, chi tiêu bao nhiêu, tên nhân viên
 // và tên trong báo cáo Sandbox -> để soi vì sao Chi phí QC = 0.

```

app.get('/api/salary/debug', async (req, res) => {
  const since = req.query.since || '2026-05-01', until = req.query.until || '2026
  const days = listDays(since, until);
  const taiKhoan = [];
  for (const src of SOURCES) {
    for (const acc of src.accounts) {
      try {
        const camps = await fetchAccount(acc, src.token, days, since, until);
        const spendOf = c => (c.daily || []).reduce((a, d) => a + (+d.spent || 0)
        const emp = {};
        for (const c of camps) { const e = c.employee || '(không nhận ra)'; emp[e
        taiKhoan.push({ acc, ok: true, soCampaign: camps.length, tongChiTieu: Mat
        } catch (e) { taiKhoan.push({ acc, ok: false, loi: e.message }); }
      }
    }
  }
  let tenTrongBaoCao = [];
  try { if (!sandboxCookie) await sandboxLogin(); tenTrongBaoCao = reportRowsEx(a
  res.json({ since, until, soNguonToken: SOURCES.length, taiKhoan, tenTrongBaoCao
});

```

// ===== Dữ liệu NHẬP TAY cho bảng lương (lưu theo tháng + nhân viên) =====
 // Cấu trúc: { name, channels:{ thailan:{dt,qc,gv,ship}, pushsale:{...}, san:{...
 // luongCung, thuong, phat, bxxh }
 //
 // QUAN TRỌNG: Dữ liệu lưu ở thư mục NGOÀI project (DATA_DIR) để KHÔNG bị mất
 // khi deploy/ghi đè thư mục nodejs. __dirname = .../ads.tdmjsc.com/nodejs


```

// nên ../../../../data = /home/u422036594/data. Có thể đổi qua .env DATA_DIR.
const MANUAL_FILE = path.join(DATA_DIR, 'salary-manual.json');
const OLD_MANUAL_FILE = path.join(__dirname, 'salary-manual.json');

let SALARY_MANUAL = {};
try {
  // Ưu tiên đọc dữ liệu từ DATA_DIR (chỗ an toàn)
  SALARY_MANUAL = JSON.parse(fs.readFileSync(MANUAL_FILE, 'utf8'));
  // Nếu file chính rỗng nhưng backup có data → khôi phục từ backup
  if (!Object.keys(SALARY_MANUAL).length) {
    try {
      const bak = JSON.parse(fs.readFileSync(MANUAL_FILE + '.bak', 'utf8'));
      if (Object.keys(bak).length) { SALARY_MANUAL = bak; fs.writeFileSync(MANUAL
    } catch {}
  }
} catch {
  // File chính lỗi → thủ backup, rồi file cũ trong project
  try {
    const bak = JSON.parse(fs.readFileSync(MANUAL_FILE + '.bak', 'utf8'));
    if (Object.keys(bak).length) { SALARY_MANUAL = bak; console.log('[RESTORE] sa
  } catch {
    try {
      SALARY_MANUAL = JSON.parse(fs.readFileSync(OLD_MANUAL_FILE, 'utf8'));
      fs.writeFileSync(MANUAL_FILE, JSON.stringify(SALARY_MANUAL));
    } catch { SALARY_MANUAL = {}; }
  }
}
function saveManual() {
  try {
    // — MERGE: đọc file đĩa, gộp RAM vào, rồi mới ghi —
    // Nếu RAM thiếu tháng nào (do restart mất data) → tháng đó trên đĩa vẫn còn
    let disk = {};
    try { disk = JSON.parse(fs.readFileSync(MANUAL_FILE, 'utf8')) || {}; } catch

    // Gộp: mỗi tháng+nhân viên trong RAM ghi đè vào disk,
    // nhưng các tháng/nhân viên CHỈ có trên đĩa được GIỮ NGUYÊN
    for (const [month, emps] of Object.entries(SALARY_MANUAL)) {
      if (!disk[month]) disk[month] = {};
      for (const [key, val] of Object.entries(emps || {})) {
        disk[month][key] = val;
      }
    }

    // Backup: chỉ ghi .bak nếu .bak chưa tồn tại HOẶC file đĩa hiện tại
    // có NHIỀU data hơn .bak (tránh ghi đè backup tốt bằng data xấu)
    const diskSize = JSON.stringify(disk).length;
    let bakSize = 0;

```

```

    try { bakSize = fs.statSync(MANUAL_FILE + '.bak').size; } catch {}
    if (diskSize >= bakSize) {
      try { fs.writeFileSync(MANUAL_FILE + '.bak', JSON.stringify(disk)); } catch {}
    }

    // Ghi file chính + đồng bộ RAM
    fs.writeFileSync(MANUAL_FILE, JSON.stringify(disk));
    SALARY_MANUAL = disk;
  } catch (e) { console.error('[SAVE] salary-manual lỗi:', e.message); }
}

const SALARY_CHANNELS = ['thailan', 'pushsale', 'san'];
const CH_METRICS = ['dt', 'qc', 'gv', 'ship'];
const numClean = v => { const n = Math.round(Number(String(v == null ? 0 : v).replace(/,/g, ''))); return n; }

// Lấy dữ liệu tay theo tháng: /api/salary/manual?month=YYYY-MM
app.get('/api/salary/manual', (req, res) => {
  const month = req.query.month || '';
  const raw = SALARY_MANUAL[month] || {};
  // Điền BHXH mặc định từ EMPLOYEES cho từng nhân viên chưa có hoặc = 0
  const data = {};
  for (const [k, v] of Object.entries(raw)) {
    const empDef = EMPLOYEES.find(e => normProd(e.full) === k);
    const bxxhDefault = empDef ? (empDef.bxxh || 0) : 0;
    data[k] = { ...v, bxxh: (v.bxxh !== null && v.bxxh !== 0) ? v.bxxh : bxxhDefault };
  }
  // Thêm BHXH mặc định cho nhân viên chưa có record trong tháng này
  for (const emp of EMPLOYEES) {
    if (!emp.bxxh) continue;
    const k = normProd(emp.full);
    if (!data[k]) data[k] = { name: emp.full, channels: {}, lươngCung: 0, thưởng: 0 };
    else if (!data[k].bxxh) data[k].bxxh = emp.bxxh;
  }
  res.json({ month, channels: SALARY_CHANNELS, data });
});

// Xem + khôi phục backup dữ liệu nhập tay
app.get('/api/salary/manual/backup', (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false });
  const files = [
    { name: 'salary-manual', path: MANUAL_FILE },
    { name: 'salary-product-manual', path: PTSP_MANUAL_FILE },
    { name: 'salary-snapshots', path: SNAPSHOT_FILE },
    { name: 'salary-published', path: PUBLISH_FILE },
  ];
  const info = files.map(f => {
    let main = null, bak = null;
    try { const j = JSON.parse(fs.readFileSync(f.path, 'utf8')); main = { keys: 0

```

```

        try { const j = JSON.parse(fs.readFileSync(f.path + '.bak', 'utf8')); bak = {
            return { name: f.name, main, bak };
        });
        res.json({ ok: true, files: info });
    });
    app.post('/api/salary/manual/restore', express.json(), (req, res) => {
        const me = req.session.user || {};
        if (me.role !== 'admin') return res.status(403).json({ ok: false });
        const { file } = req.body || {};
        const map = { 'salary-manual': MANUAL_FILE, 'salary-product-manual': PTSP_MANUA
        const p = map[file];
        if (!p) return res.json({ ok: false, message: 'File không hợp lệ' });
        try {
            const bak = fs.readFileSync(p + '.bak', 'utf8');
            const parsed = JSON.parse(bak);
            fs.writeFileSync(p, bak);
            // Nạp lại vào RAM
            if (file === 'salary-manual') SALARY_MANUAL = parsed;
            else if (file === 'salary-product-manual') PTSP_MANUAL = parsed;
            else if (file === 'salary-snapshots') SNAPSHOTS = parsed;
            else if (file === 'salary-published') PUBLISHED = parsed;
            res.json({ ok: true, keys: Object.keys(parsed).length, message: 'Đã khôi phục
        } catch (e) { res.json({ ok: false, message: 'Không có backup hoặc lỗi: ' + e.m
    });

// Lưu 1 nhân viên
app.post('/api/salary/manual', (req, res) => {
    const { month, name, values } = req.body || {};
    if (!month || !name) return res.status(400).json({ ok: false, message: 'Thiếu t
    const k = normProd(name);
    if (!SALARY_MANUAL[month]) SALARY_MANUAL[month] = {};
    const v = values || {};
    // Điền BHXH mặc định từ EMPLOYEES nếu tháng này chưa có giá trị
    const empDef = EMPLOYEES.find(e => normProd(e.full) === normProd(name));
    const bhxhDefault = empDef ? (empDef.bhxh || 0) : 0;
    const out = { name, channels: {}, lươngCung: numClean(v.lươngCung), thưởng: num
    for (const ch of SALARY_CHANNELS) {
        out.channels[ch] = {};
        const src = (v.channels && v.channels[ch]) || {};
        for (const m of CH_METRICS) out.channels[ch][m] = numClean(src[m]);
    }
    SALARY_MANUAL[month][k] = out;
    saveManual();
    res.json({ ok: true });
});

/* ===== CÔNG KHAI LƯƠNG QUA TELEGRAM =====

```

- Gửi bảng lương riêng cho từng nhân viên qua Telegram.
- Điều kiện: lương tháng X chỉ gửi được SAU ngày 10 tháng X+1.
- Cần TELEGRAM_BOT_TOKEN trong .env và Chat ID trong users.js.

```

===== */
const TELEGRAM_BOT_TOKEN = process.env.TELEGRAM_BOT_TOKEN || '';

// Lưu lịch sử công khai (tháng nào đã gửi) trong DATA_DIR
const PUBLISH_FILE = path.join(DATA_DIR, 'salary-published.json');
let PUBLISHED = {};
try { PUBLISHED = JSON.parse(fs.readFileSync(PUBLISH_FILE, 'utf8')); } catch { }
function savePublished() {
  try {
    const newJson = JSON.stringify(PUBLISHED, null, 0);
    let oldJson = '{}';
    try { oldJson = fs.readFileSync(PUBLISH_FILE, 'utf8'); } catch {}
    if (oldJson !== '{}') {
      try { fs.writeFileSync(PUBLISH_FILE + '.bak', oldJson); } catch {}
    }
    fs.writeFileSync(PUBLISH_FILE, newJson);
  } catch (e) {}
}

// Snapshot bảng lương đã chốt – lưu để nhân viên xem mà KHÔNG cần tính lại
const SNAPSHOT_FILE = path.join(DATA_DIR, 'salary-snapshots.json');
let SNAPSOTS = {};
try { SNAPSOTS = JSON.parse(fs.readFileSync(SNAPSHOT_FILE, 'utf8')); } catch { }
function saveSnapshots() {
  try {
    const newJson = JSON.stringify(SNAPSOTS, null, 0);
    let oldJson = '{}';
    try { oldJson = fs.readFileSync(SNAPSHOT_FILE, 'utf8'); } catch {}
    if (oldJson !== '{}') {
      try { fs.writeFileSync(SNAPSHOT_FILE + '.bak', oldJson); } catch {}
    }
    fs.writeFileSync(SNAPSHOT_FILE, newJson);
  } catch (e) {}
}

// Kiểm tra đã qua ngày 10 của tháng SAU tháng lương chưa
function canPublish(monthStr) {
  // monthStr = 'YYYY-MM' (tháng lương)
  const [y, m] = monthStr.split('-').map(Number);
  // Mốc: ngày 10 của tháng kế tiếp
  const unlock = new Date(y, m, 10); // m vì Date month 0-based → m = tháng kế ti
  const now = new Date();
  return now >= unlock;
}

```

```

// Gửi 1 tin nhắn Telegram
async function sendTelegram(chatId, text) {
  if (!TELEGRAM_BOT_TOKEN) throw new Error('Chưa cấu hình TELEGRAM_BOT_TOKEN');
  const url = `https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/sendMessage`;
  const r = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ chat_id: chatId, text, parse_mode: 'HTML' }),
  });
  const j = await r.json().catch(() => ({}));
  return { ok: r.ok && j.ok, error: j.description };
}

const fmtVnd = n => (Math.round(Number(n) || 0)).toLocaleString('vi-VN');

// Soạn nội dung lương Marketing cho 1 nhân viên
function buildMktMessage(row, monthStr, extra) {
  const e = extra || {};
  const thuc = (row.luong || 0) + (e.hoaHongLeader || 0) + (e.luongCung || 0) + (
    let t = `<b>💰 BẢNG LƯƠNG THÁNG ${monthStr}</b>\n`;
    t += `Nhân viên: <b>${row.name}</b>\n`;
    t += `—————\n`;
    t += `Doanh thu: <b>${fmtVnd(row.doanhthu)}</b> đ\n`;
    t += `Chi phí QC: ${fmtVnd(row.chiPhiQC)} đ\n`;
    t += `Giá vốn: ${fmtVnd(row.giaVon)} đ\n`;
    t += `Phí ship: ${fmtVnd(row.phiShip)} đ\n`;
    t += `—————\n`;
    t += `Lương 2%: <b>${fmtVnd(row.luong)}</b> đ\n`;
    if (e.hoaHongLeader) t += `Hoa hồng Leader: ${fmtVnd(e.hoaHongLeader)} đ\n`;
    if (e.luongCung) t += `Lương cúng: ${fmtVnd(e.luongCung)} đ\n`;
    if (e.thuong) t += `Thưởng: ${fmtVnd(e.thuong)} đ\n`;
    if (e.phat) t += `Phạt: -${fmtVnd(e.phat)} đ\n`;
    if (e.bhxx) t += `BHXH: -${fmtVnd(e.bhxx)} đ\n`;
    t += `—————\n`;
    t += `<b>💰 THỰC NHẬN: ${fmtVnd(thuc)} đ</b>`;
    return t;
  }

}

// API công khai lương Marketing: nhận danh sách rows đã tính từ client
app.post('/api/salary/publish', express.json({ limit: '2mb' }), async (req, res) {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chỉ
  const { month, rows } = req.body || {};
  if (!month || !Array.isArray(rows)) return res.json({ ok: false, message: 'Thiếu
  if (!canPublish(month)) {
    const [y, m] = month.split('-').map(Number);

```

```

    return res.json({ ok: false, message: `Chưa đến hạn công khai. Lương tháng ${
    }
// Lưu snapshot để nhân viên xem trên web (KHÔNG gửi Telegram)
SNAPSHOTS['mkt-' + month] = { at: new Date().toISOString(), month, type: 'mkt',
saveSnapshots();
PUBLISHED['mkt-' + month] = { at: new Date().toISOString(), count: rows.length
savePublished();
res.json({ ok: true, count: rows.length });
});

// Soạn nội dung lương PTSP cho 1 người
function buildPtspMessage(row, monthStr) {
    const thuc = (row.hoaHong || 0) + (row.luongCung || 0) + (row.thuong || 0) - (r
    let t = `📄 BẢNG LƯƠNG PTSP THÁNG ${monthStr}</b>\n`;
    t += `Nhân viên: <b>${row.manager}</b>\n`;
    t += `_____ \n`;
    t += `Số SP: ${fmtVnd(row.soSP)} (mới ${fmtVnd(row.soSPmoi)}, cũ ${fmtVnd(row.s
    t += `Đơn ship: ${fmtVnd(row.soDon)} · SL: ${fmtVnd(row.soLuongSP)} \n`;
    t += `Doanh thu: <b>${fmtVnd(row.doanhThu)}</b> đ \n`;
    t += `Giá vốn: ${fmtVnd(row.giaVon)} đ \n`;
    t += `_____ \n`;
    t += `Hoa hồng: <b>${fmtVnd(row.hoaHong)}</b> đ \n`;
    if (row.luongCung) t += `Lương cứng: ${fmtVnd(row.luongCung)} đ \n`;
    if (row.thuong) t += `Thuởng: ${fmtVnd(row.thuong)} đ \n`;
    if (row.phat) t += `Phạt: -${fmtVnd(row.phat)} đ \n`;
    if (row.bhxx) t += `BHXH: -${fmtVnd(row.bhxx)} đ \n`;
    t += `_____ \n`;
    t += `💰 THỰC NHẬN: ${fmtVnd(thuc)} đ</b>`;
    return t;
}

// API công khai lương PTSP
app.post('/api/salary-product/publish', express.json({ limit: '2mb' }), async (re
    const me = req.session.user || {};
    if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chi
    const { month, rows } = req.body || {};
    if (!month || !Array.isArray(rows)) return res.json({ ok: false, message: 'Thiế
    if (!canPublish(month)) {
        const [y, m] = month.split('-').map(Number);
        return res.json({ ok: false, message: `Chưa đến hạn công khai. Lương tháng ${
    }
// Lưu snapshot để nhân viên xem trên web (KHÔNG gửi Telegram)
SNAPSHOTS['ptsp-' + month] = { at: new Date().toISOString(), month, type: 'ptsp
saveSnapshots();
PUBLISHED['ptsp-' + month] = { at: new Date().toISOString(), count: rows.length
savePublished();
res.json({ ok: true, count: rows.length });

```

```

});

/* ===== API CHO NHÂN VIÊN XEM LƯƠNG ĐÃ CÔNG KHAI (từ snapshot) ===== */
// Trả về danh sách tháng đã công khai mà nhân viên này có dữ liệu
app.get('/api/my-salary/months', (req, res) => {
  const me = req.session.user || {};
  // salaryName = tên để xem LƯƠNG (chỉ của chính mình). Nếu không có thì fallback
  const myNames = me.salaryName ? [me.salaryName]
    : (me.role === 'product' && me.manager ? [me.manager]
      : (me.employees && me.employees.length ? [me.employees[0]] : []));
  const months = [];
  for (const [key, snap] of Object.entries(SNAPSHOTS)) {
    const isMine = (snap.rows || []).some(r => {
      const n = r.name || r.manager;
      return myNames.includes(n);
    });
    if (isMine) months.push({ key, month: snap.month, type: snap.type, at: snap.a
  }
  months.sort((a, b) => b.month.localeCompare(a.month));
  res.json({ ok: true, months, myNames });
});

// Trả về bảng lương của chính nhân viên trong 1 tháng đã công khai
app.get('/api/my-salary/detail', (req, res) => {
  const me = req.session.user || {};
  const key = req.query.key || '';
  const snap = SNAPSHOTS[key];
  if (!snap) return res.json({ ok: false, message: 'Chưa có bảng lương công khai
  const myNames = me.salaryName ? [me.salaryName]
    : (me.role === 'product' && me.manager ? [me.manager]
      : (me.employees && me.employees.length ? [me.employees[0]] : []));
  // Admin xem được tất cả; nhân viên chỉ thấy dòng của mình
  let rows = snap.rows || [];
  if (me.role !== 'admin') {
    rows = rows.filter(r => myNames.includes(r.name || r.manager));
  }
  res.json({ ok: true, month: snap.month, type: snap.type, at: snap.at, rows });
});

// Xem trạng thái cache Meta (admin) - biết đã cache những khoảng nào
app.get('/api/meta-cache/status', (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ error: 'Chỉ admin' });
  const list = Object.entries(META_CACHE).map(([key, v]) => ({
    range: key, at: v.at, soChienDich: (v.campaigns || []).length,
    daGhim: !!v.pinned, ghimLuc: v.pinnedAt || null, ghimBoi: v.pinnedBy || null,
    tongNganSach: (v.campaigns || []).reduce((s, c) => s + (Number(c.budget) || 0

```

```

    })).sort((a, b) => b.range.localeCompare(a.range));
    res.json({
      ok: true,
      soKhoang: list.length,
      soKhoangDaGhim: list.filter(x => x.daGhim).length,
      list,
    });
  });
});

// Kiểm tra 1 khoảng đã ghim chưa (để dashboard đổi trạng thái nút)
app.get('/api/meta-cache/is-pinned', (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ error: 'Chỉ admin' });
  const key = (req.query.since || '') + '|' + (req.query.until || '');
  const rec = META_CACHE[key];
  res.json({
    ok: true,
    daGhim: !!(rec && rec.pinned),
    coCache: !!rec,
    ghimLuc: rec && rec.pinnedAt || null,
    soChienDich: rec ? (rec.campaigns || []).length : 0,
  });
});

// Xoá cache 1 khoảng hoặc tất cả (admin) - khi muốn lấy lại số liệu mới từ Faceb
app.post('/api/meta-cache/clear', express.json(), (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ error: 'Chỉ admin' });
  const { range, includePinned } = req.body || {};
  const kept = [];
  if (range === 'all') {
    for (const k of Object.keys(META_CACHE)) {
      if (META_CACHE[k].pinned && !includePinned) { kept.push(k); continue; }
      delete META_CACHE[k];
    }
    clearRamCache();
  } else if (range && META_CACHE[range]) {
    if (META_CACHE[range].pinned && !includePinned)
      return res.json({ ok: false, daGhim: true, message: 'Khoảng này đã được ghi' });
    delete META_CACHE[range]; DATA_CACHE.delete(range);
  }
  else if (range) { DATA_CACHE.delete(range); } // chưa có cache vĩnh viễn nhun
  else return res.json({ ok: false, message: 'Không tìm thấy khoảng cần xoá' });
  saveMetaCache();
  res.json({ ok: true, giữ_lại_vì_đã_ghim: kept.length, cac_key_da_ghim: kept });
});

```


LƯU Ý VỀ "TỔNG NGÂN SÁCH": lấy CHI TIÊU THỰC TẾ trên Meta (đã +11% VAT), không phải "ngân sách/ngày" cấu hình trên Facebook. Ngân sách/ngày chỉ là mức trần cài đặt, không phải tiền thật đã tiêu → không dùng cho P&L được.

△ Công thức lương MKT/PTSP dưới đây PHẢI KHỚP với salary.html và salary-product.html. Nếu sau này đổi quy tắc lương ở 2 trang đó, nhớ sửa cả ở đây.

```
const THUE_SUAT = Number(process.env.THUE_SUAT || 0.17);    // 17% thuế TNDN

// Chi phí + lương nhập tay theo tháng
// { luongSale, luongKeToan, luongKho, chiPhiList:[{ten, soTien}] }
const KQKD_FILE = path.join(DATA_DIR, 'ket-qua-kinh-doanh.json');
let KQKD_MANUAL = {};
try { KQKD_MANUAL = JSON.parse(fs.readFileSync(KQKD_FILE, 'utf8')) || {}; } catch
function saveKqkd() {
  try { fs.writeFileSync(KQKD_FILE, JSON.stringify(KQKD_MANUAL)); }
  catch (e) { console.error('[SAVE] ket-qua-kinh-doanh lỗi:', e.message); }
}
const KQKD_FIELDS = ['luongSale', 'luongKeToan', 'luongKho'];

// Lấy bản ghi 1 tháng, tự nâng cấp dữ liệu cũ (chiPhi là 1 số) sang danh sách
function kqkdOf(month) {
  const r = KQKD_MANUAL[month] || (KQKD_MANUAL[month] = {});
  if (!Array.isArray(r.chiPhiList)) {
    r.chiPhiList = (+r.chiPhi) ? [{ ten: 'Chi phí khác', soTien: +r.chiPhi }] : [
      delete r.chiPhi;
    ]
  }
  return r;
}

/* GET /api/business-result?month=YYYY-MM
  Trả DỮ LIỆU THÔ (báo cáo lương MKT + PTSP + số nhập tay).
  Việc TÍNH LƯƠNG do salary-calc.js làm ở trình duyệt – dùng chung với
  trang Lương MKT / Lương PTSP nên số liệu luôn khớp nhau tuyệt đối.    */
app.get('/api/business-result', async (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chi
  const month = req.query.month || new Date().toISOString().slice(0, 7);
  const [y, mo] = month.split('-').map(Number);
  if (!y || !mo) return res.json({ ok: false, message: 'Tháng không hợp lệ' });
  const since = `${y}-${String(mo).padStart(2, '0')}-01`;
  const until = `${y}-${String(mo).padStart(2, '0')}-${String(new Date(y, mo, 0).
  try {
```

```

const [repMKT, repPTSP] = await Promise.all([
  buildSalaryReport(since, until),
  buildPtspReport(since, until, { role: 'admin' }),
]);
if (!repMKT.ok) return res.json({ ok: false, message: 'Lương MKT: ' + (repMKT
if (!repPTSP.ok) return res.json({ ok: false, message: 'Lương PTSP: ' + (repP

res.json({
  ok: true, month, since, until,
  thueSuat: THUE_SUAT,
  mkt: { rows: repMKT.rows, roster: repMKT.roster, tyLe: repMKT.tyLe, teamLea
  mktManual: SALARY_MANUAL[month] || {},
  ptsp: { managers: repPTSP.managers },
  ptspManual: PTSP_MANUAL[month] || {},
  tay: kqkdOf(month),
  lastUpdated: new Date().toISOString(),
});
} catch (e) {
  const msg = e.name === 'AbortError' ? 'API Sandbox không phản hồi (timeout).
  res.json({ ok: false, month, message: msg });
}
});

/* Danh sách chi phí: thêm / sửa / xóa
  POST /api/business-result/chi-phi {month, action:'add'|"update"|"delete', inde
app.post('/api/business-result/chi-phi', express.json(), (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chi
  const { month, action, index, ten, soTien } = req.body || {};
  if (!month) return res.json({ ok: false, message: 'Thiếu tháng' });
  const rec = kqkdOf(month);
  const money = v => parseInt(String(v == null ? '' : v).replace(/^[^d-]/g, ''),
  const i = Number(index);

  if (action === 'add') {
    const name = String(ten || '').trim();
    if (!name) return res.json({ ok: false, message: 'Nhập tên loại chi phí' });
    rec.chiPhiList.push({ ten: name, soTien: money(soTien) });
  } else if (action === 'update') {
    if (!(i >= 0 && i < rec.chiPhiList.length)) return res.json({ ok: false, mess
    if (ten != null) rec.chiPhiList[i].ten = String(ten).trim();
    if (soTien != null) rec.chiPhiList[i].soTien = money(soTien);
  } else if (action === 'delete') {
    if (!(i >= 0 && i < rec.chiPhiList.length)) return res.json({ ok: false, mess
    rec.chiPhiList.splice(i, 1);
  } else {
    return res.json({ ok: false, message: 'action phải là add / update / delete'

```

```

    }
    saveKqkd();
    res.json({ ok: true, chiPhiList: rec.chiPhiList });
  });

// Lưu số nhập tay: POST /api/business-result/manual {month, field, value}
app.post('/api/business-result/manual', express.json(), (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chỉ
const { month, field, value } = req.body || {};
  if (!month || !KQKD_FIELDS.includes(field))
    return res.json({ ok: false, message: 'Thiếu tháng hoặc field không hợp lệ' });
  const rec = kqkdOf(month);
  rec[field] = parseInt(String(value == null ? '' : value).replace(/^[^d-]/g, ''))
  saveKqkd();
  res.json({ ok: true, month, data: rec });
});

app.get('/api/business-result/manual', (req, res) => {
  const me = req.session.user || {};
  if (me.role !== 'admin') return res.status(403).json({ ok: false, message: 'Chỉ
  res.json({ ok: true, data: kqkdOf(req.query.month || '') });
});

app.use(express.static(path.join(__dirname, 'public')));
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Đang chạy: http://localhost:${PORT}`));

```